

MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL SUL-RIO-GRANDENSE
CÂMPUS PELOTAS
CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET



Pedro Lucas Porcellis

Nexo — aplicativo de monitoramento do humor para pessoas com transtorno bipolar

Trabalho de Conclusão de Curso

Pelotas
2026

Pedro Lucas Porcellis

Nexo — aplicativo de monitoramento do humor para pessoas com transtorno bipolar

Trabalho de Conclusão de Curso apresentado como requisito parcial à obtenção ao título de tecnólogo em Sistemas para Internet do Instituto Federal Sul-Rio-Grandense

Orientador: Prof. Me. Flávio Nunes

Pelotas
2026

Insira AQUI a ficha catalográfica
(solicitada na página da biblioteca)

PEDRO LUCAS PORCELLIS

NEXO — APLICATIVO DE MONITORAMENTO DO HUMOR PARA PESSOAS COM TRANSTORNO BIPOLAR

Trabalho de Conclusão de Curso apresentado como requisito parcial à obtenção ao título de tecnólogo em Sistemas para Internet do Instituto Federal Sul-Rio-Grandense

Orientador: Prof. Me. Flávio Nunes

Aprovado pela banca examinadora em: ___/___/_____

Prof. Me. Flávio Nunes - IFSul
Prof.º Orientador

Prof. Dr. Fulano de Souza e Silva - UFI
Membro titular

Prof. Dr. Sicrano Silveira Rosa - UFPG
Membro titular

Prof. Dr. Beltrano Pereira Tavares - UFCR
Membro titular

Texto da Dedicatória

AGRADECIMENTOS

Texto dos Agradecimentos

“Texto da Epígrafe”
Autor da Epígrafe

RESUMO

Este trabalho apresenta o desenvolvimento do Nexo, uma aplicação móvel de código aberto voltada ao monitoramento do humor de pessoas com Transtorno Bipolar, fundamentada nos princípios da *Ecological Momentary Assessment* (EMA). O objetivo foi conceber uma ferramenta que integrasse, em uma única plataforma, o registro de humor, a qualidade do sono, gatilhos e ações de cuidado, de modo a auxiliar o usuário na identificação de padrões afetivos ao longo do tempo. A pesquisa caracteriza-se como aplicada, de natureza exploratória, e combinou levantamento bibliográfico, aplicação de questionário com potenciais usuários e análise de aplicativos similares, empregando o modelo incremental e princípios ágeis com *Kanban* no desenvolvimento. Como resultado, obteve-se um protótipo funcional composto por uma API REST e um aplicativo móvel, ambos em TypeScript, com processamento de indicadores em segundo plano, implantação automatizada e testes automatizados da API. Conclui-se que os objetivos propostos foram alcançados, evidenciando a viabilidade de uma solução de autorregistro alinhada à privacidade e à autonomia do usuário sobre seus dados.

Palavras-chave: Ecological Momentary Assessment, Transtorno Bipolar, Monitoramento de humor, React Native, TypeScript, PostgreSQL.

ABSTRACT

This work presents the development of Nexo, an open-source mobile application for mood monitoring aimed at people with Bipolar Disorder, grounded in the principles of the Ecological Momentary Assessment (EMA). The goal was to design a tool that integrated, within a single platform, the recording of mood, sleep quality, triggers, and care actions, in order to help users identify affective patterns over time. The research is applied and exploratory in nature, combining a literature review, a questionnaire applied to potential users, and an analysis of similar applications, while adopting an incremental development model and agile principles with Kanban. As a result, a functional prototype was produced, composed of a REST API and a mobile application, both written in TypeScript, with background processing of indicators, automated deployment, and automated tests for the API. It is concluded that the proposed objectives were achieved, demonstrating the feasibility of a self-tracking solution aligned with user privacy and autonomy over their data.

Keywords: Ecological Momentary Assessment, Bipolar Disorder, Mood Tracker, React Native, TypeScript, PostgreSQL.

LISTA DE ILUSTRAÇÕES

Figura 1 Quadro de tarefas do projeto 21
Figura 2 Aplicativo Baseline 26
Figura 3 Aplicativo DailyBeans 27
Figura 4 Aplicativo DailyMood 28
Figura 5 Aplicativo Reflectly 29
Figura 6 Aplicativo eMoods 31
Figura 7 Diagrama de Casos de Uso 40
Figura 8 Diagrama de Classes 42
Figura 9 Logotipo do sistema 45
Figura 10 Paleta de cores 47
Figura 11 Referência da fonte Inter 48
Figura 12 *Wireframe* da tela inicial 50
Figura 13 *Wireframe* da tela de indicadores 50
Figura 14 Arquitetura e elementos do Sistema 59
Figura 15 Tela de login 61
Figura 16 Tela de Consulta de Registros 62
Figura 17 Criação de Registros de Humor 63
Figura 18 Adicionando sentimentos/emoções 64
Figura 19 Vinculando um humor com um gatilho 65
Figura 20 Avaliando o impacto de um gatilho no humor 66
Figura 21 Tela de indicadores 67
Figura 22 Tela de Histórico 69
Figura 23 Tela de geração de relatório 72

LISTA DE TABELAS

Tabela 1 Síntese dos resultados do questionário 24
Tabela 2 Comparativo de funcionalidades entre aplicativos analisados 32
Tabela 3 Requisitos funcionais 36
Tabela 4 Requisitos não funcionais 37
Tabela 5 Vocabulário de testes adotado neste trabalho 75
Tabela 6 Ferramentas de verificação 76

LISTA DE CÓDIGOS

Código 1 Validação e Inferência usando Zod 53
Código 2 Configuração do *healthcheck* do Valkey e de dependência condicional da API no `docker-compose.yml` 56
Código 3 Formato comum da tela de histórico 70

LISTA DE ABREVIATURAS E SIGLAS

TAB	Transtorno Afetivo Bipolar
BAP	Bipolar Affective Disorder
TCC	Terapia Cognitivo-comportamental
EMA	*Ecological Momentary Assessment*
UML	*Unified Modeling Language*
DAO	*Data Access Object*
REST	*Representational State Transfer*
API	*Application Programming Interface*
UX	*User Experience*
SSH	*Secure Shell*
CI	*Continuous Integration*
CD	*Continuous Deployment*

SUMÁRIO

1 INTRODUÇÃO 15

2 SAÚDE E AUTOCUIDADO MENTAL NO MUNDO DIGITAL 17

3 METODOLOGIAS 20
3.1 METODOLOGIA DE DESENVOLVIMENTO DE *SOFTWARE* 20
3.2 METODOLOGIA DE PESQUISA 22
3.2.1 Tipo de Pesquisa 22
3.2.2 Instrumento de coleta de dados 23
3.2.3 Análise de Similares 25
3.2.3.1 Baseline 26
3.2.3.2 DailyBeans 26
3.2.3.3 DailyMood 28
3.2.3.4 Reflectly 29
3.2.3.5 eMoods 30
3.2.4 Análise comparativa 31
3.2.5 Relação entre os dados do questionário e análise de similares 32

4 NEXO 34
4.1 CLASSIFICAÇÃO DOS REQUISITOS 35
4.1.1 Requisitos funcionais 35
4.1.2 Requirementos não funcionais 37
4.2 MODELAGEM 39
4.2.1 Casos de Uso 39
4.2.2 Diagrama de Classes 41

5 DESIGN 44
5.1 IDENTIDADE VISUAL 44
5.1.1 Nome e Logotipo 45
5.1.2 Hierarquia visual 46
5.1.3 Paleta de Cores 46
5.1.4 Tipografia 47
5.2 ESTRUTURA 48
5.2.1 Organização da navegação 48
5.2.2 Consolidação de registros no Histórico 49
5.2.3 Wireframes 49

6 TECNOLOGIAS 51
6.1 FRONT-END 51
6.1.1 React Native 51
6.1.2 Zustand 51
6.2 BACK-END 52
6.2.1 Express.js 52
6.2.2 Zod 52
6.2.3 Valkey e BullMQ 53
6.3 INFRAESTRUTURA E PERSISTÊNCIA DE DADOS 54
6.3.1 PostgreSQL 54
6.3.2 Prisma 55
6.4 IMPLANTAÇÃO E DESENVOLVIMENTO CONTÍNUO 55
6.4.1 Docker e Docker Compose 55
6.4.2 GitHub Actions 56
6.4.3 Ansible 57

7 ARQUITETURA E FUNCIONAMENTO DO SISTEMA 59
7.1 PROCESSO DE LOGIN 59
7.2 PÁGINA DE CONSULTA DE REGISTROS 61
7.3 CRIAÇÃO DE REGISTROS DE HUMOR 63
7.4 INDICADORES 66
7.5 HISTÓRICO 68
7.6 EXPORTAÇÃO DE DADOS E RELATÓRIOS 70

8 TESTES 74
8.1 VOCABULÁRIO 74
8.2 ESTRATÉGIA DE VERIFICAÇÃO 75
8.3 SUÍTE DE TESTES DA API 76
8.3.1 Organização e ferramentas 76
8.3.2 Execução e banco de dados de teste 77
8.3.3 Tipos de testes 77
8.3.3.1 Testes unitários 77
8.3.3.2 Testes de integração 78
8.3.3.3 Testes de tarefas assíncronas 78
8.4 VERIFICAÇÃO DO APLICATIVO MÓVEL 78
8.4.1 Suíte *end-to-end* (Maestro) 79
8.5 INTEGRAÇÃO CONTÍNUA 79
8.6 COBERTURA E LACUNAS 80

9 CONSIDERAÇÕES FINAIS 81

REFERÊNCIAS 83

APÊNDICE A – DOCUMENTAÇÃO DE CASOS DE USO 88

APÊNDICE B – RESULTADOS DA PESQUISA 97

1 INTRODUÇÃO

O avanço da tecnologia digital tem transformado profundamente a forma como indivíduos interagem com sua saúde mental, com seus hábitos cotidianos e com os recursos disponíveis para o autocuidado. Em um cenário de sobrecarga de informações e de crescimento dos quadros de ansiedade e depressão, cresce também a busca por ferramentas que auxiliem no reconhecimento e na gestão das emoções de forma autônoma e personalizada.

Esse cenário é particularmente relevante para o Transtorno Afetivo Bipolar (TAB), condição caracterizada por flutuações cíclicas do humor que tornam difícil, para o próprio indivíduo, identificar em que etapa do ciclo se encontra. O autorregistro contínuo — aproximando a coleta do momento em que a experiência ocorre, nos moldes do *Ecological Momentary Assessment* — é uma via consolidada para reduzir os vieses de memória e favorecer o desenvolvimento de consciência afetiva e de autoconhecimento.

No entanto, as aplicações disponíveis para esse fim ainda apresentam lacunas: frequentemente fragmentam sono, humor, gatilhos e ações de cuidado em telas isoladas, restringem a exportação dos dados e operam como soluções fechadas, com pouca transparência sobre o tratamento das informações sensíveis que coletam. Essas limitações evidenciam a necessidade de uma solução que concentre os registros e preserve a autonomia do usuário sobre os próprios dados.

Diante dessas lacunas, este trabalho tem como objetivo desenvolver o Nexo, uma aplicação móvel de código aberto voltada ao acompanhamento do humor de pessoas com TAB, capaz de produzir indicadores que auxiliem o usuário na percepção de padrões afetivos ao longo do tempo. Para construí-la, o desenvolvimento apoia-se em levantamento bibliográfico, na aplicação de um questionário a potenciais usuários e na análise de aplicativos similares para levantar requisitos, e culmina na modelagem, na implementação e na verificação automatizada do sistema.

Além desta introdução, o documento está organizado em oito capítulos. O Capítulo 2 revisa a literatura sobre transtorno bipolar, autorregistro e *Ecological Momentary Assessment*, situando as limitações dos aplicativos existentes. O Capítulo 3 apresenta as metodologias de desenvolvimento e de pesquisa adotadas, incluindo o questionário aplicado e a análise de aplicativos similares. O Capítulo 4 descreve o

Nexo, seus requisitos e a modelagem do sistema. O Capítulo 5 trata do design visual e da estrutura de navegação. O Capítulo 6 apresenta as tecnologias empregadas. O Capítulo 7 descreve a arquitetura e o funcionamento das principais funcionalidades. O Capítulo 8 discute a estratégia de verificação adotada. Por fim, o Capítulo 9 apresenta as considerações finais e as melhorias futuras.

2 SAÚDE E AUTOCUIDADO MENTAL NO MUNDO DIGITAL

No quesito de saúde mental, a bipolaridade ou pelo seu nome científico, Transtorno Afetivo Bipolar (TAB), é uma das doenças mentais mais comuns e responsável por uma gama muito grande de disfunções mentais. De acordo com o estudo conduzido por MERIKANGAS et al. (2011), no âmbito do *World Mental Health Survey Initiative* da Organização Mundial da Saúde, com mais de 61 mil adultos entrevistados em 11 países, a prevalência ao longo da vida do espectro bipolar é de 2,4% da população mundial, e a prevalência em um período de 12 meses é de 1,5%. No contexto do Brasil, segundo a Associação Brasileira de Transtorno Bipolar (ABTB), capítulo brasileiro da *International Society for Bipolar Disorders* (ISBD) (Associação Brasileira de Transtorno Bipolar (ISBD Brazil), 2026), a bipolaridade afeta cerca de 4% da população, o que representaria uns 7,6 milhões de pessoas. Esse número, embora soe pouco expressivo, levando em consideração o contingente de pessoas no país, é preocupante, dado o fato de que a bipolaridade é (1) complexa de se detectar e (2) muitas vezes, acaba passando despercebida ou enquadrada dentro de um quadro de depressão unipolar (depressão).

Embora o autorregistro seja amplamente recomendado no acompanhamento de pessoas com TAB, dado seu potencial de validade clínica e de efeito sobre desfechos relevantes (FAURHOLT-JEPSEN et al., 2016), sua realização de maneira contínua representa um desafio. Em muitos casos, o paciente depende da própria memória para reconstruir retrospectivamente seu estado emocional, a qualidade do sono, acontecimentos cotidianos e outros fatores que podem ter influenciado seu humor. Esse processo está sujeito, naturalmente, a vieses de recordação e dificulta a identificação de padrões temporais, especialmente quando as informações são coletadas apenas durante consultas clínicas.

Nesse contexto, o *Ecological Momentary Assessment* (EMA) consolidou-se como uma importante abordagem metodológica para coleta de dados em saúde mental. Diferentemente de avaliações retrospectivas, o EMA propõe que informações sejam registradas no ambiente natural do indivíduo e próximas ao momento em que as experiências ocorrem, reduzindo vieses de memória e permitindo observar a dinâmica dos estados afetivos ao longo do tempo (EBNER-PRIEMER; TRULL, 2009). Além de proporcionar maior validade ecológica aos dados coletados, essa estratégia tem sido

empregada em diversas áreas da pesquisa comportamental, incluindo transtornos do humor (COLLINS; KASHDAN; GOLLNISCH, 2003; EBNER-PRIEMER; TRULL, 2009), permitindo compreender a relação entre fatores contextuais, eventos cotidianos e alterações do humor.

Nesse contexto, o uso de tecnologias digitais tem se mostrado uma via promissora para sistematizar esse tipo de acompanhamento. Pesquisas indicam que pessoas com transtornos do espectro bipolar demonstram alta adesão ao monitoramento emocional quando este é feito de forma acessível, visual e integrada ao seu cotidiano, com estudos reportando taxas de conclusão acima de 90% em protocolos de avaliação momentânea via *smartphone* (RYAN et al., 2020) [1].

Entre as formas mais comuns de implementação do EMA encontra-se o autorregistro contínuo (*journaling*), no qual o indivíduo registra sistematicamente aspectos relevantes do cotidiano, como humor, qualidade do sono, eventos significativos, sintomas percebidos e outras informações contextuais. Com o acúmulo desses registros, torna-se possível identificar padrões de comportamento, reconhecer fatores associados às alterações de humor e favorecer o desenvolvimento da autoconsciência sobre a própria condição, constituindo uma ferramenta complementar ao processo de autoconhecimento e autogestão do comportamento (MELTON; BIGHAM; BLAND, 2013).

A ampla disseminação dos dispositivos móveis ampliou significativamente as possibilidades de utilização dessas estratégias fora do ambiente clínico. Smartphones acompanham o indivíduo durante praticamente toda sua rotina, permitindo que registros sejam realizados no momento em que as experiências ocorrem e reduzindo o esforço necessário para manutenção do acompanhamento contínuo. Como consequência, diversas aplicações voltadas ao monitoramento da saúde mental passaram a incorporar princípios do EMA em seus fluxos de utilização, oferecendo registros estruturados e visualizações históricas capazes de auxiliar usuários e profissionais de saúde (FAURHOLT-JEPSEN et al., 2016).

Apesar da crescente disponibilidade de aplicações voltadas ao acompanhamento do humor, estudos recentes que avaliaram especificamente os aplicativos mais utilizados por pessoas com TAB para monitoramento de humor e sono apontam limitações importantes quanto à integração entre esses domínios, ao controle do usuário sobre seus próprios dados e à acessibilidade das políticas de privacidade, muitas vezes redigidas em nível de leitura incompatível com o público geral (MORTON et al., 2022).

Em paralelo, análises mais amplas do ecossistema de aplicativos de saúde mental também identificam riscos recorrentes de privacidade, como permissões desnecessárias e vazamento de dados sensíveis (IWAYA et al., 2022). Essas limitações

[1] Vale citar porém, que esse artigo tem limitações de generalização: os resultados são limitados a um grupo de pessoas que possuiam um maior contato com tecnologia.

evidenciam a necessidade de soluções que conciliem boas práticas de engenharia de software, proteção de dados pessoais e recursos que apoiem a identificação de padrões emocionais ao longo do tempo.

Diante desse contexto, este trabalho propõe o desenvolvimento de uma aplicação móvel fundamentada nos princípios do *Ecological Momentary Assessment* voltada ao acompanhamento cotidiano de pessoas com Transtorno Afetivo Bipolar. O sistema busca integrar registros estruturados de humor, qualidade do sono, gatilhos e ações de cuidado em uma única plataforma, produzindo indicadores que auxiliem o usuário na visualização de padrões e no desenvolvimento da consciência sobre seu próprio estado afetivo. Ressalta-se que a aplicação não possui finalidade diagnóstica nem substitui o acompanhamento realizado por profissionais de saúde, sendo concebida como uma ferramenta de apoio ao autorregistro e ao autocuidado.

3 METODOLOGIAS

Como primeiro passo, para conseguir viabilizar construir um *software* com qualidade, é necessário adotar metodologias que fundamentam a ideia e o embasem o processo. De tal sorte que, ajudem a levantar quais requisitos esse *software* deve atender, priorizar funcionalidades, quebrar o escopo em entregáveis que possam ser validados, e desenvolver os mesmos de forma sustentável. Esse capítulo se debruça, tanto na abordagem da metodologia de pesquisa do projeto, quanto também detalha o método de desenvolvimento de *software* escolhido.

3.1 METODOLOGIA DE DESENVOLVIMENTO DE *SOFTWARE*

Existem diversas abordagens para o desenvolvimento de *software*, cada uma com seus respectivos prós e contras. Todos procuram responder a uma mesma necessidade: dar forma ao processo de desenvolvimento, para que ele flua e assegure que os requisitos levantados sejam atendidos, que os membros envolvidos no processo, sejam eles os desenvolvedores quanto os usuários finais ou atores que o demandam tenham seus anseios respondidos de forma minimamente transparente e satisfatória.

Além disso, a escolha de uma abordagem adequada para as necessidades do sistema, permite que os prazos de entrega sejam mais bem controlados e mais precisos (PRESSMAN, 2021), minimizando a ocorrência de falhas e melhorando a gestão do tempo.

As ideias fundamentais que norteiam esse projeto são baseadas em dois conceitos chaves da engenharia de *software*: o modelo de desenvolvimento incremental e desenvolvimento de *software* ágil. O IEEE define que a engenharia de *software* é:

> Engenharia de *software*: A aplicação de uma abordagem sistemática, disciplinada e quantificável no desenvolvimento, na operação e na manutenção de *software*; isto é, a aplicação de engenharia ao *software*. (IEEE, 1990, p. 67, tradução nossa)

Portanto, a engenharia de *software* propõe estruturar a criação de *software* de forma mais organizada e eficiente. PRESSMAN (2021) aborda, entre os diversos modelos de processo, PRESSMAN (2021) aborda o de desenvolvimento em cascata e desenvolvimento incremental que moldaram boa parte das práticas da área nas

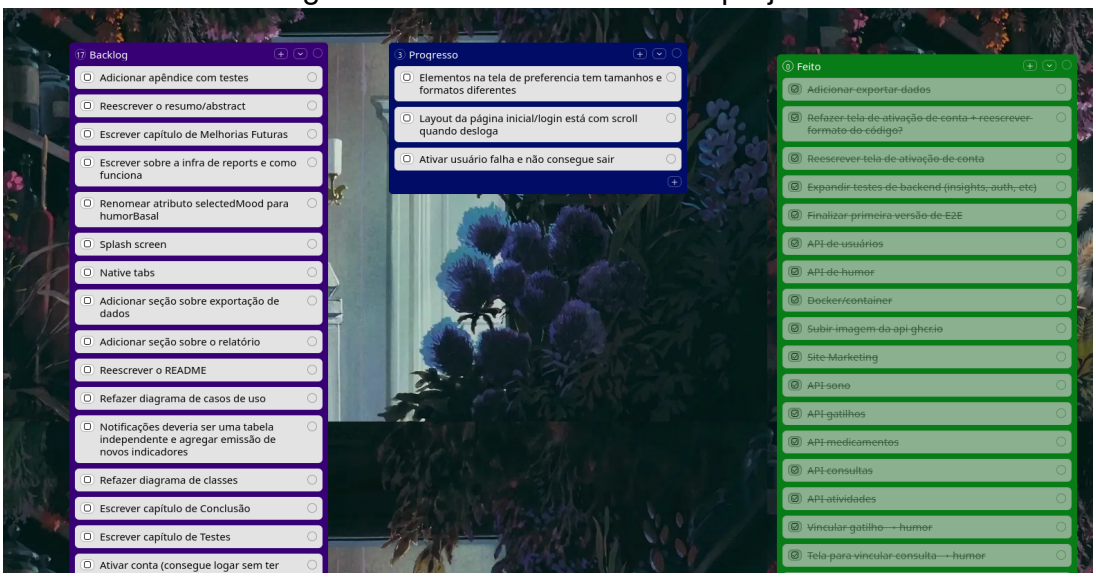
últimas décadas.

O modelo em cascata (*waterfall*) é linear e um dos mais tradicionais. Levantamento de requisitos, projeto, implementação e testes formam uma única linha reta e única, na qual cada etapa precisa estar concluída antes da entrega final. Como não há entrega intermediária, o retorno sobre o produto só é coletado depois que ele está pronto. Embora forneça uma estrutura lógica e rígida, o modelo é frequentemente criticado pela baixa adaptabilidade em relação às mudanças durante o processo de desenvolvimento.

Em outro oposto, o modelo incremental, trabalha com a premissa de construir em pequenas porções. Cada incremento percorre seu próprio mini-ciclo de levantamento de requisitos, projeto, implementação e testes, e ao fim de cada ciclo curto, entrega-se um sistema funcional, ainda que parcial. Isso permite colher retorno precocemente, lidar melhor com requisitos incertos ou mutáveis e ajustar o rumo a cada entrega (PRESSMAN, 2021).

Pressman, comenta que, por meados de 2001, um grupo de desenvolvedores renomados publicou o “Manifesto Ágil”, um documento desenhando um conjunto de valores/princípios de forma a explicitar elementos chaves do ciclo de desenvolvimento, destacando a satisfação dos *stakeholders* do sistema, entregas mais rápidas, flexibilidade durante o desenvolvimento do *software* de forma a evitar que os desenvolvedores ficassem empacados e ajudasse a remover a rigidez de modelos prescritivos.

Figura 1: Quadro de tarefas do projeto



Fonte: Elaborado pelo autor (2026)

Por serem princípios, e não procedimentos, os valores ágeis não são aplicados diretamente: precisam ser instanciados em métodos concretos, como o *Kanban* e o *Scrum*. O *Scrum* organiza o trabalho em papéis e cerimônias (*Product Owner*, *Scrum*

Master, *dailies*, *sprint reviews*) pensados para coordenar uma equipe. O *Kanban*, por sua vez, baseia-se na visualização do fluxo de trabalho, como demonstrado na Figura 1 e na limitação de tarefas em progresso, sem pressupor uma estrutura de equipe (ANDERSON, 2010).

Por ser um método não prescritivo, o *Kanban* não define um ciclo de vida de desenvolvimento; ele organiza a execução do trabalho definido por outro modelo. Isso permite que as tarefas sejam priorizadas, ajustadas ou redefinidas, conforme a evolução do projeto, característica especialmente útil quando há apenas um desenvolvedor responsável por todas as etapas do processo.

Com base nesse panorama, o presente trabalho adotou um misto: utiliza o modelo incremental estrutura o ciclo de desenvolvimento em entregas funcionais sucessivas; os princípios ágeis orientam as decisões, priorizando em entregas funcionais sucessivas; os princípios ágeis orientam as decisões, priorizando a entrega contínua de valor e a adaptação a mudanças; e o *Kanban* organiza a execução por meio de um quadro que separa o que precisa ser feito, o que está em andamento e o que já foi concluído, limitando a quantidade de tarefas em progresso. Essa combinação mantém o controle sobre o andamento e a qualidade do produto sem os entraves de cerimônias e papéis pensados para equipes, o que a torna adequada a um desenvolvimento individual.

3.2 METODOLOGIA DE PESQUISA

Entende-se por pesquisa científica, a investigação de um fenômeno, de forma a encontrar a solução para algum problema (COELHO, 2020). Portanto, a metodologia científica se torna o conjunto de procedimentos para esta investigação, cujo qual perpassa a coleta e análise de dados. Neste capítulo, abordaremos as metodologias adotadas e suas respectivas justificativas, para além dos dados encontrados para tal.

3.2.1 Tipo de Pesquisa

Segundo (LAKATOS; MARCONI, 2022), a pesquisa científica pode ser classificada quanto aos seus objetivos como exploratória, descritiva ou explicativa. Este trabalho caracteriza-se principalmente como uma pesquisa aplicada de natureza exploratória, uma vez que busca investigar práticas, limitações e necessidades relacionadas a ferramentas digitais para acompanhamento emocional, com o objetivo de desenvolver uma solução tecnológica prática.

Conforme (GERHARDT; SILVEIRA, 2009), a pesquisa aplicada visa gerar conhecimento para aplicação direta em problemas específicos, com interesse prático e utilitário, sendo comum em projetos de desenvolvimento de *software*, especialmente nas áreas de Sistemas de Informação e Ciência da Computação, nas quais o produto

final visa resolver uma demanda concreta do público-alvo. Nesse contexto, a coleta de dados junto a potenciais usuários orienta as decisões do projeto, conforme detalhado a seguir.

3.2.2 Instrumento de coleta de dados

Como instrumento de coleta de dados, optou-se pela aplicação de um questionário estruturado, elaborado com o intuito de compreender hábitos, preferências e resistências de usuários em relação ao uso de aplicativos voltados para o acompanhamento emocional e práticas de autorreflexão.

A escolha do questionário se justifica pela sua abrangência, agilidade na coleta de dados e facilidade de análise quantitativa e qualitativa. Além disso, como a amostra é composta por usuários potenciais, a aplicação do questionário de forma digital por meio de um formulário online permite alcançar um público mais amplo com baixo custo e maior comodidade (EVANS; MATHUR, 2005).

O questionário, disponível no Apêndice B, foi divulgado de forma aberta entre 7 e 16 de julho de 2025 e obteve 48 respostas. Trata-se, portanto, de uma amostra por conveniência, cujos resultados descrevem o grupo respondente e não podem ser generalizados. A amostra é composta majoritariamente por mulheres (32 respostas; 66,7%), com idades entre 18 e 30 anos (21; 43,8%), 31 e 45 anos (14; 29,2%), 46 e 60 anos (11; 22,9%) e acima de 60 anos (2; 4,2%). Dos participantes, 18 (37,5%) relataram diagnóstico relacionado à saúde mental. Como a divulgação foi aberta e o tame envolve saúde mental e experiências subjetivas, a primeira pergunta “Você já teve algum interesse ou experiência com o monitoramento do seu humor ou estado emocional?”, buscava situar os respondentes quanto ao perfil de potencias usuários.

Nessa pergunta, apenas 4 participantes (8,3%) afirmaram não ter interesse nem experiência, enquanto 25 (52,1%) afirmaram ter interesse, mas nunca ter tentado. Na pergunta específica sobre histórico, 26 dos 47 respondentes (55,3%) já monitoraram o humor em algum momento da vida, de forma regular, ocasional ou interrompida. A Tabela 1 sintetiza os resultados mais relevantes para as decisões do projeto.

Tabela 1: Síntese dos resultados do questionário

Pergunta	**Resposta**	**Freq./base**	**%**
Interesse ou experiência com monitoramento de humor	“Tenho interesse, mas nunca tentei”	25/48	52,1
Já monitorou o humor em algum momento da vida	Sim (regular, ocasional ou interrompido)	26/47	55,3
Já recebeu diagnóstico relacionado à saúde mental	Sim	18/48	37,5
Motivo mais citado para abandono ou dificuldade[1]	Esquecimento	14/25	56,0
Segundo motivo mais citado[1]	Falta de tempo	10/25	40,0
Terceiro motivo mais citado[1]	Preocupações com privacidade	6/25	24,0
Importância de ter controle total sobre os próprios dados	Nota máxima (5)	32/47	68,1
Preferência de armazenamento	Apenas no dispositivo (sem nuvem)	14/48	29,2
Conforto em compartilhar dados anonimamente para pesquisa	Sim	28/48	58,3
Tom de comunicação preferido	“Acolhedor e empático”	18/48	37,5
Utilidade de o aplicativo avisar sobre possíveis ciclos de humor	Sim	36/48	75,0

[1] Resposta de múltipla escolha; base: 25 respondentes que relataram motivos (excluídas respostas como “nunca fiz”).

Fonte: Elaborado pelo autor (2026), dados de 2025

Quanto à detecção de padrões, 33 participantes (68,8%) acreditam que um aplicativo pode ajudar a perceber padrões emocionais que normalmente não notariam, e outros 13 (27,1%) responderam “talvez”. Dois participantes mencionaram espontaneamente a visualização gráfica da evolução do humor, como em “seria legal ver um gráfico semanal ou mensal do meu humor”. Quanto aos ciclos de humor, 36 participantes (75,0%) consideraram útil que o aplicativo os identifique e avise, e 10 (20,8%) condicionaram a utilidade à precisão. Contudo, quando avaliada como funcionalidade em escala de 1 a 5, a detecção de ciclos obteve média 2,93, com 31,8% de notas 4 ou

5 (considerando total de 44 respondentes). O resultado indica interesse no conceito, mas com cautela quanto à sua execução, o que reforça a necessidade de comunicar com clareza a incerteza das inferências.

Em relação à privacidade, 32 dos 47 respondentes (68,1%) atribuíram a nota máxima à importância de ter controle total sobre os próprios dados, e a privacidade foi o terceiro motivo mais citado para abandono (24,0%). Essa percepção está alinhada com a literatura sobre saúde digital: IWAYA et al. (2022) argumentam que a ausência de mecanismos transparentes de consentimento, controle e anonimização pode minar a confiança dos usuários em aplicativos de bem-estar, especialmente em contextos sensíveis como saúde mental.

Ao mesmo tempo, não houve consenso sobre o armazenamento: 29,2% preferem guardar os dados apenas no dispositivo, 22,9% aceitam a nuvem criptografada, 22,9% são indiferentes desde que possam acessar e exportar, e 25,0% não tem preferência. Já 58,3% se sentiriam confortáveis em compartilhar dados de forma anônima para pesquisa, e 29,2% responderam que dependeria do tipo de dado.

Por fim, quanto ao tom, o “acolhedor e empático” foi o mais frequente (37,5%), seguido do “neutro e objetivo” (25,0%), do “clínico e profissional” (20,8%) e do “lúdico e leve” (12,5%). Embora o tom acolhedor se destaque, a ausência de preferência majoritária indica que a linguagem do aplicativo deve ser cuidadosa, sem pressupor um único registro. Em relação à frequência, 20 participantes (41,7%) afirmaram conseguir registrar o humor uma vez ao dia e 12 (25,0%), duas vezes, o que sustenta um registro leve e recorrente.

3.2.3 Análise de Similares

A metodologia trata-se de uma avaliação sobre os sistemas relacionados ao tema proposto, observando e levantando desafios e possibilidades, validação de *softwares* e a satisfação dos usuários (Martins, 2018). Seguindo assim, foram pesquisados sistemas similares encontrados na internet com o objetivo de realizar uma revisão crítica-reflexiva, analisando os aplicativos que oferecem algum tipo de diário digital de humor, ou permite algum acompanhamento reflexivo de variações de emoções.

Para realizar esse processo, de forma a simular também um usuário, foi pesquisado na Google Play Store [1], por *mood tracker*, retornando centenas de resultados. Nesse quesito, estabeleceu os seguintes critérios objetivos: (1) pode ser um aplicativo em português ou inglês; (2) o aplicativo tem que apresentar algum tipo de gráfico das entradas feitas (3) tem que possuir entrada textual.

[1] Google Play Store — Disponível em: https://play.google.com

3.2.3.1 Baseline

O Baseline é um aplicativo de código aberto voltado ao acompanhamento do bem-estar emocional por meio de uma abordagem minimalista. A ferramenta permite o registro diário de humor utilizando entradas textuais ou gravações de áudio, além de disponibilizar notificações personalizadas e revisões periódicas dos registros realizados.

Figura 2: Aplicativo Baseline



Fonte: Elaborado pelo autor (2026)

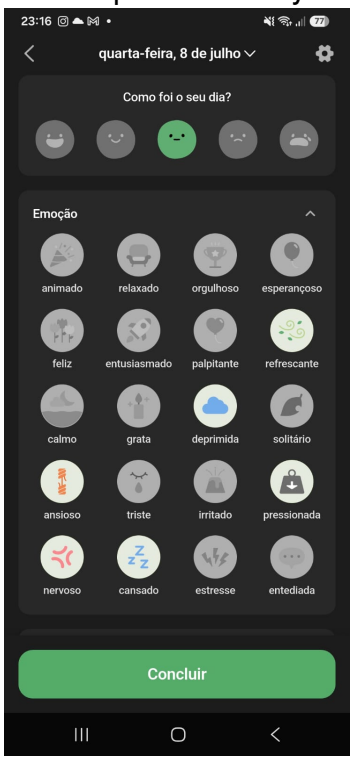
Entre os aspectos observados, destaca-se a natureza open-source da aplicação, característica que favorece a transparência e a possibilidade de adaptação por terceiros. Sua interface apresenta baixo nível de complexidade e reduz a quantidade de interações necessárias para o registro das informações. Entretanto, nota-se que o sistema não oferece visualizações temporais avançadas nem mecanismos de personalização mais aprofundados. Além disso, o acompanhamento emocional é realizado principalmente por meio de uma pontuação de humor, sem identificar ciclos emocionais ou relacionar registros a gatilhos e intervenções específicas.

3.2.3.2 DailyBeans

O DailyBeans é um dos aplicativos de monitoramento de humor mais populares disponíveis atualmente, possuindo como principal característica a simplicidade de uso. Nota-se que possui uma linguagem bem lúdica e informal, por exemplo, o sistema uti-

liza o conceito de “feijões” representando o estado de humor do usuário. Destaca-se que o aplicativo permite selecionar categorias de sentimentos e adicionar atividades para preencher melhor o registro do estado emocional do usuário para além de um “Feliz”, “Triste”, etc. Existe também uma bastante vasta opções de categorias, cada qual com uma ilustração que representa o que aquilo representa. O aplicativo também apresenta “Temas sazonais”, permitindo que o usuário customize cores, icones, e demais informações. Um elemento interssante é que as informações adicionadas também podem ser *offlines*, até uma certa quantidade de entradas/registros, necessitando de uma conta para poder continuar salvando os dados informados.

Figura 3: Aplicativo DailyBeans



Fonte: Elaborado pelo autor (2026)

Entre suas funcionalidades, além do registro de humor e categorização de atividades e sentimentos, o aplicativo conta com gráficos e históricos. Uma interface bastante coesa e visualmente agradável. Um outro elemento é que o aplicativo permite relacionar indicadores de sono com bastante precisão, como poder indicar o momento que foi dormir, momento que acordou, anotações, etc. O aplicativo permite adicionar pelo menos uma foto no plano gratuito, além de indicar elementos como “Qual o clima do dia”, se o usuário esteve com “amigos”, “familiares”.

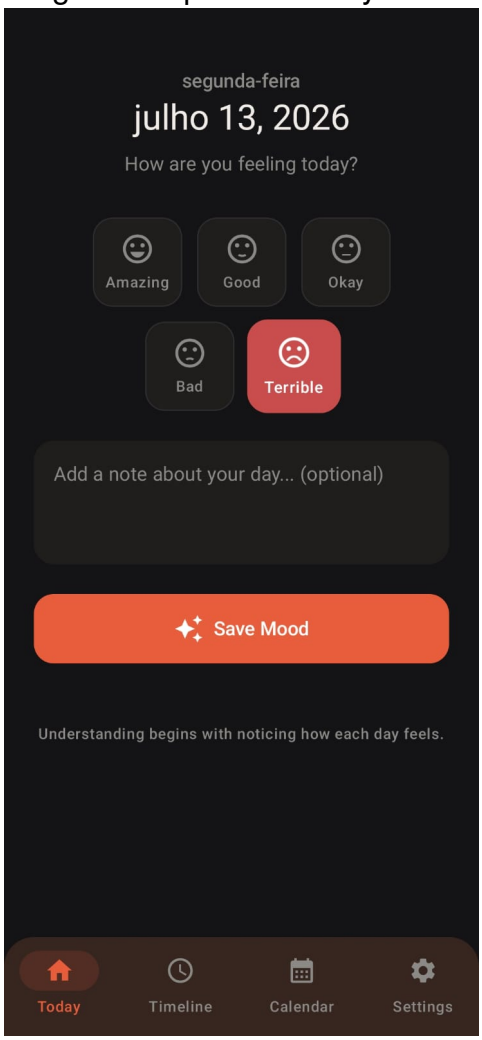
Observa-se que o aplicativo possui código fechado e restringe parte de suas funcionalidades mais avançadas à versão paga ou compras na Loja interna do aplicativo para obter por exemplo, personalização de icones, adicionar mais fotos. Além

disso, o foco em registros rápidos reduz a flexibilidade para inserção de reflexões textuais mais detalhadas, e uma exibição excessante de anúncios atrapalha a utilização.

3.2.3.3 DailyMood

DailyMood é um aplicativo voltado ao registro cotidiano do estado emocional, oferecendo uma solução enxuta para acompanhamento de humor. Sua proposta baseia-se em uma interação mínima, permitindo que o usuário selecione rapidamente o sentimento predominante do dia, sem a necessidade de descrições extensas. Os registros são organizados em um calendário, o que facilita a leitura histórica e a identificação de padrões emocionais ao longo do tempo.

Figura 4: Aplicativo DailyMood



Fonte: Elaborado pelo autor (2026)

Apesar dos recursos disponíveis, observa-se que é uma solução bastante simples. Não existe outros recursos além de informar o humor predominante e uma anotação. Um fator interessante é que não necessita de contas, rodando completamente

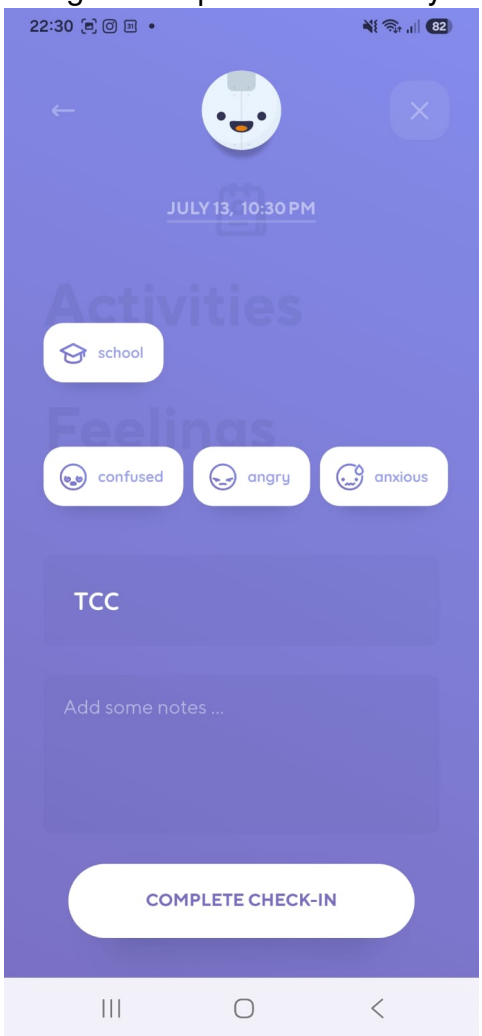
offline no dispositivo do usuário. Adicionalmente, apresenta limitações relacionadas à exportação dos dados registrados e à personalização das informações armazenadas.

3.2.3.4 Reflectly

O Reflectly é um aplicativo de diário pessoal que utiliza recursos de inteligência artificial para promover práticas de autorreflexão e acompanhamento emocional. Sua proposta enfatiza a experiência do usuário e a construção de narrativas pessoais a partir dos registros realizados.

Entre suas funcionalidades estão entradas textuais livres, perguntas guiadas diárias, análises emocionais automatizadas e personalização visual da interface. O sistema apresenta uma experiência moderna e intuitiva, incentivando o uso contínuo por meio de elementos de engajamento. Um ponto interessante é o fato do aplicativo separar o “humor” dos “sentimentos”, permitindo que os usuários avaliem ambos de forma independente.

Figura 5: Aplicativo Reflectly



Fonte: Elaborado pelo autor (2026)

Como pontos observados, destaca-se o fato de ser uma solução proprietária e dependente de coleta significativa de dados para alimentar seus mecanismos de análise. Outro ponto observado é o fato de que o aplicativo apresenta elementos como estatísticas e histórico, porém necessita de uma conta paga para acessar esses recursos.

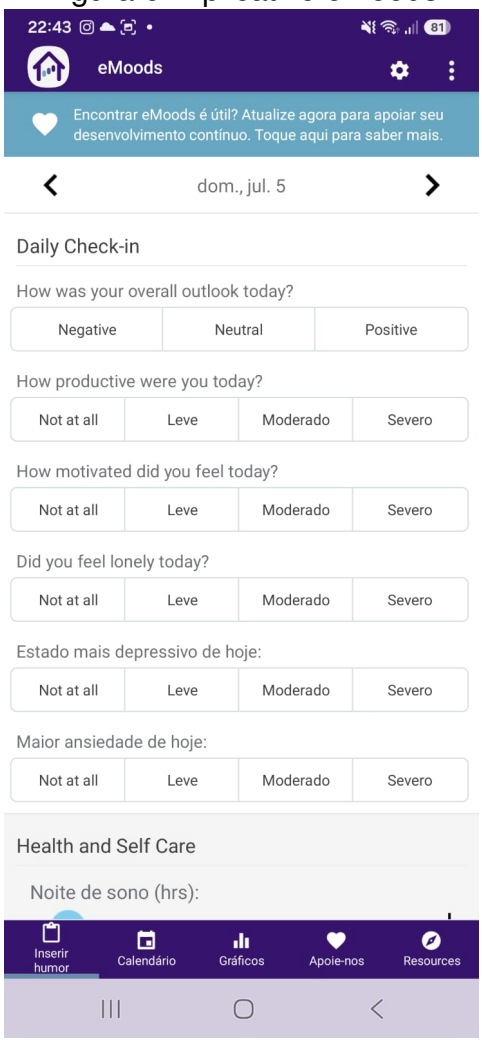
Além disso, sua abordagem está mais voltada ao diário pessoal do que ao acompanhamento sistemático de indicadores de saúde mental.

3.2.3.5 eMoods

O *eMoods* é um aplicativo de registro de humor especificamente para pessoas com TAB. O aplicativo tem uma interface bem compacta, bastante poluída visualmente, em estilo formulário de entrada de dados.

O aplicativo consegue registrar dados de sono, além de perguntas como ansiedade, estado depressivo, e outras coisas como “você praticou higiene pessoal?”, “você tomou medicamentos?”, “você se alimentou bem?” e afins. Um ponto destacado é que o aplicativo também permite exportar os dados em formato CSV e gerar relatórios em formato PDF.

Figura 6: Aplicativo eMoods



Fonte: Elaborado pelo autor (2026)

Entretanto, a aplicação apresenta uma experiência do usuário um tanto confusa, possuindo interface bastante poluída quando comparada a soluções mais novas/contemporâneas. Também não oferece suporte para registros textuais extensos ou entradas por voz, além de simular um formulário.

3.2.4 Análise comparativa

Coletando essa lista de aplicações, é possível então dispor as funcionalidades mais relevantes de cada aplicação da seguinte forma:

Tabela 2: Comparativo de funcionalidades entre aplicativos analisados

Funcionalidade	Baseline	DailyBeans	DailyMood	Reflectly	eMoods
Registro de humor	Sim	Sim	Sim	Sim	Sim
Registro de sono	Não	Não	Não	Não	Sim
Texto livre	Sim	Parcial	Sim	Sim	Sim
Gráficos	Não	Sim	Sim	Sim	Sim
Gatilhos/intervenções	Não	Parcial	Não	Não	Parcial
Exportação de dados	Sim	Parcial	Parcial	Parcial	Sim
Código aberto	Sim	Não	Não	Não	Não

Fonte: Elaborado pelo autor (2026)

Com esses dados em mente, podemos compreender alguns elementos fundamentais: poucos sistemas analisados são *open-source*, o que indica que, em sua grande maioria, são sistemas comerciais. A maior parte dos sistemas também oferecem algum aspecto de pagamento, bloqueando funcionalidades importantes atrás de algum tipo de modelo de subscrição. Existe também o fato de personalização e de exportação de dados, que poucos ou nenhum sistema oferece, o que faz com que o sistema não tenha portabilidade dos dados inseridos, criando assim, um *walled garden*, ou um jardim fechado — uma técnica de fazer com que o usuário não consiga migrar entre sistemas, prendendo-o a usar aquela aplicação já que seus dados estão lá.

3.2.5 Relação entre os dados do questionário e análise de similares

A análise dos sistemas similares, quando considerada em conjunto com os dados coletados por meio do questionário, revela lacunas e oportunidades relevantes para o desenvolvimento de uma nova solução. Observa-se, por exemplo, que a maior parte das ferramentas disponíveis no mercado segue modelos comerciais, frequentemente com funcionalidades essenciais bloqueadas por sistemas de pagamento ou assinaturas. Além disso, é comum que tais sistemas sejam fechados, sem disponibilização de código-fonte ou possibilidade de exportação dos dados inseridos, o que configura uma prática de *walled garden*, dificultando a migração entre plataformas e restringindo a autonomia dos usuários sobre suas próprias informações.

Esses aspectos, quando contrastados com o interesse expressivo dos respondentes por visualizar padrões emocionais ao longo do tempo e por maior personalização, sugerem direções claras para o desenvolvimento de um sistema que não apenas atenda a essas demandas, mas também adote princípios de transparência, abertura e controle por parte do usuário. A seguir, serão discutidas as decisões de projeto que nortearam a construção da ferramenta proposta, baseadas tanto nas limitações identificadas nos sistemas existentes quanto nas necessidades e expectativas manifestadas

pelo público-alvo da pesquisa.

4 NEXO

O aplicativo proposto por esse trabalho, denominado “Nexo”, se propõe a ser uma aplicação para dispositivos móveis, de forma a preencher essas lacunas demonstradas tanto pelo questionário, quanto também tentando preencher as questões analisadas em outros sistemas similares. Como mencionado anteriormente, o sistema é direcionado para pessoas com TAB, embora, ele também possa atuar isoladamente como um aplicativo de registro de humor, e portanto, o nome reflete essa escolha. A lógica é, de um aplicativo que consegue conectar diversos pontos, que em um primeiro momento parecem soltos: sono, rotina, práticas, humor, gatilhos.

A partir das demandas identificadas na pesquisa e dos princípios estabelecidos para este projeto, delineia-se um sistema de monitoramento de humor centrado no usuário, com ênfase em personalização, privacidade e utilidade reflexiva. O funcionamento do sistema parte da premissa de que o humor não é uma variável universal, mas sim composto por elementos subjetivos que variam entre indivíduos. Assim, fundamentalmente para cada entrada no diário, o usuário pode indicar que componentes de humor ele percebe em vigência naquele momento, como “raiva”, “tristeza”, “felicidade”, etc., elementos que ele considera mais representativos de seu estado emocional, permitindo um mapeamento mais fiel da sua experiência.

O monitoramento, pode ser gerado em diversos momentos do dia, o que torna o aplicativo mais flexível, ao invés de uma única entrada no dia. O usuário pode também anotar para cada momento, algo relevante, através de escrita livre, e usar uma escala que possa indicar o que ele percebe como indicadores dos níveis de energia, estresse e ansiedade. Associado a isso, o sistema permitirá a entrada de gatilhos e eventos significativos, sejam positivos ou negativos, que poderão ser posteriormente correlacionados com os estados de humor, permitindo identificar padrões ou ciclos. Esses dados podem ser correlacionados/vinculados com “práticas” ou “intervenções”, como fármacos, psicoterapia, atividades físicas.

Outro ponto essencial é a exportação de dados em formatos acessíveis, como o formato CSV (SHAFRANOVICH, 2005) [1], garantindo que o usuário tenha controle e mobilidade sobre suas informações, algo frequentemente negligenciado em sistemas

[1] Formato de arquivo onde cada coluna é separado por um delimitador, por exemplo: vírgulas, ponto-e-vírgula, apresentado no RFC 4180

fechados. Além disso, a ferramenta é por inteiro *software* livre, garantindo transparência de funcionamento, possibilidade de auditoria e fomento a melhorias comunitárias, especialmente importantes no contexto de tecnologias sensíveis à saúde mental. Essas características formam a base de um sistema comprometido com o bem-estar, o respeito à individualidade e a soberania dos dados pessoais.

Com base nas funcionalidades listadas, pode-se definir os requisitos funcionais e não funcionais no qual o sistema deve necessitar.

4.1 CLASSIFICAÇÃO DOS REQUISITOS

Como observado por (SOMMERVILLE, 2010, p. 58), “Os requisitos de um sistema são as descrições dos serviços que o sistema deve prestar e as restrições à sua operação”. Portanto, requisitos são declarações articuladas descrevendo de forma clara o que o sistema deve ser capaz de fazer para satisfazer as necessidades postas anteriormente. São definidas em duas categorias: requisitos funcionais que descrevem o comportamento e funções do sistema e requisitos não funcionais que descrevem os critérios específicos que podem ser usados para avaliar o bom funcionamento do sistema. A separação entre requisitos funcionais e não funcionais permite uma estrutura clara para priorização, testes e validação do projeto, respeitando tanto os objetivos técnicos quanto éticos da proposta. Logo a seguir, serão abordados com mais detalhes cada um deles.

4.1.1 Requisitos funcionais

Os requisitos funcionais dizem respeito às funcionalidades centrais do sistema, ou seja, declarações de tudo o que o sistema deve e está proposto a fazer e reagir. Sommerville reforça esse aspecto:

> Os requisitos funcionais de um sistema descrevem o que ele deve fazer. Eles dependem do tipo de software a ser desenvolvido, de quem são seus possíveis usuários e da abordagem geral adotada pela organização ao escrever os requisitos. Quando expressos como requisitos de usuário, os requisitos funcionais são normalmente descritos de forma abstrata, para serem compreendidos pelos usuários do sistema.
>
> (SOMMERVILLE, 2010, p. 59)

Portanto, diante das funcionalidades previstas, que foram brevemente listadas, no Quadro 3 é apresentado os requisitos que o sistema deve fornecer:

Tabela 3: Requisitos funcionais

N°	Requisito	Descrição
[RF01]	Login	O sistema deve ser capaz de realizar login com email e senha
[RF02]	Redefinir senha	O sistema permite que o usuário redefina a sua senha, mediante informar o email usado
[RF03]	Cadastro	O sistema deve permitir que o usuário crie uma conta
[RF04]	Desconectar	O sistema permite que o usuário faça logout do sistema
[RF05]	Manter humor	O usuário deve poder registrar e atualizar seu estado emocional conforme a necessidade
[RF06]	Registrar gatilho	O sistema deve permitir associar sentimentos ou eventos específicos que influenciam o humor
[RF07]	Visualizar histórico	O usuário deve ter acesso aos registros anteriores de humor e demais tipos de dados como Gatilho, Sono, etc.
[RF08]	Visualizar detalhes do dia	O sistema deve permitir expandir e consultar dados específicos de um determinado dia
[RF09]	Visualizar registros de humor recente	O usuário deve poder visualizar os ultimos registros de humor como forma de ler
[RF10]	Manter registro de sono	O sistema deve permitir que o usuário possa cadastrar registros de sono
[RF11]	Manter ações de cuidado	O usuário deve poder cadastrar ações de cuidado (atividade, medicamentos, consultas)
[RF12]	Exportar dados	O sistema deve oferecer uma funcionalidade para exportar os dados registrados

N°	Requisito	Descrição
[RF13]	Registrar que um medicamento foi tomado	O usuário pode marcar que um determinado medicamento foi tomado mediante que foi cadastrado
[RF14]	Associar humor à um gatilho	O sistema deve permitir que o usuário vincule um humor específico com um gatilho
[RF15]	Associar consulta com um humor ou gatilho	O sistema deve permitir vincular uma consulta específica a um gatilho ou humor registrado
[RF16]	Gerar um relatório com os últimos dados	O usuário deve selecionar um período e gerar/exportar um relatório com os últimos dados

Fonte: Elaborado pelo autor (2026)

4.1.2 Requirementos não funcionais

De acordo com (SOMMERVILLE, 2010, p. 89): “Requisitos não funcionais são restrições sobre os serviços ou funções oferecidas pelo sistema”. Ou seja, definem os critérios de qualidade que o sistema deve seguir, abrangendo aspectos como segurança, usabilidade, desempenho e manutenção. Neste projeto, destacam-se especialmente a preservação da privacidade do usuário, a transparência proporcionada pelo código aberto e a possibilidade de utilização offline, o que garante acessibilidade mesmo em contextos de conectividade limitada.

Tabela 4: Requisitos não funcionais

N°	Requisito	Descrição
[RNF01]	Usabilidade	O sistema deve possuir uma interface intuitiva e acessível para usuários leigos em tecnologia, com navegação fluída e confirmações visuais após cada interação importante

N°	Requisito	Descrição
[RNF02]	Desempenho	O tempo de resposta entre a entrada do usuário e a resposta do sistema deve ser inferior a 1 segundo; O carregamento do histórico semanal e visualizações de ciclos anteriores deve ocorrer em no máximo 2 segundos.
[RNF03]	Segurança	Os dados do usuário devem ser armazenados de forma criptografada; O sistema deve exigir autenticação segura para acesso; A exportação de dados deve sempre exigir a confirmação do usuário
[RNF04]	Confiabilidade	O sistema deve garantir a persistência dos dados, mesmo em casos de falha de energia ou conexão; Os dados registrados devem ser armazenados automáticamente e sincronizados quando ouvir conexão
[RNF05]	Portabilidade	Todos os dados do usuário podem ser exportados a qualquer momento, em um formato de texto que permita que ele possa ler e interpretar, mediante compreensão mínima da linguagem/formato utilizado
[RNF06]	Privacidade	Nenhuma informação sensível do usuário deve ser compartilhada com terceiros e o sistema deve fornecer uma política de privacidade clara e acessível; Quaisquer alterações à mesma demanda que o usuário seja notificado

Fonte: Elaborado pelo autor (2026)

Com essas definições em mãos, pode-se avançar para a etapa de modelagem do sistema, usando os requerimentos apresentados como fundação para o sistema.

4.2 MODELAGEM

De acordo com (GUEDES, 2022), a modelagem de sistemas é uma etapa fundamental do processo de desenvolvimento de software, pois permite representar graficamente os principais elementos do sistema, seus comportamentos e interações. Essa representação facilita a comunicação entre desenvolvedores, usuários e demais envolvidos, servindo tanto como guia para implementação quanto como instrumento de validação dos requisitos levantados. Através da modelagem, é possível antever a estrutura do sistema, simular sua dinâmica e assegurar que ele atenderá às necessidades específicas dos usuários.

Neste subcapítulo, são apresentados dois tipos principais de diagramas da *Unified Modeling Language* (UML) empregados na modelagem do sistema proposto: o diagrama de casos de uso, que descreve as funcionalidades esperadas a partir da perspectiva do usuário; o diagrama de classes, que estrutura os elementos do domínio em termos de atributos e relacionamentos. Esses diagramas complementam-se mutuamente, proporcionando uma visão abrangente do comportamento e da arquitetura do sistema.

4.2.1 Casos de Uso

O diagrama de casos de uso é o mais geral, informal e simples de compreender (GUEDES, 2022). A base dele é o levantamento de requisitos e a pesquisa realizada, e serve como o pontapé para todo o processo de modelagem.

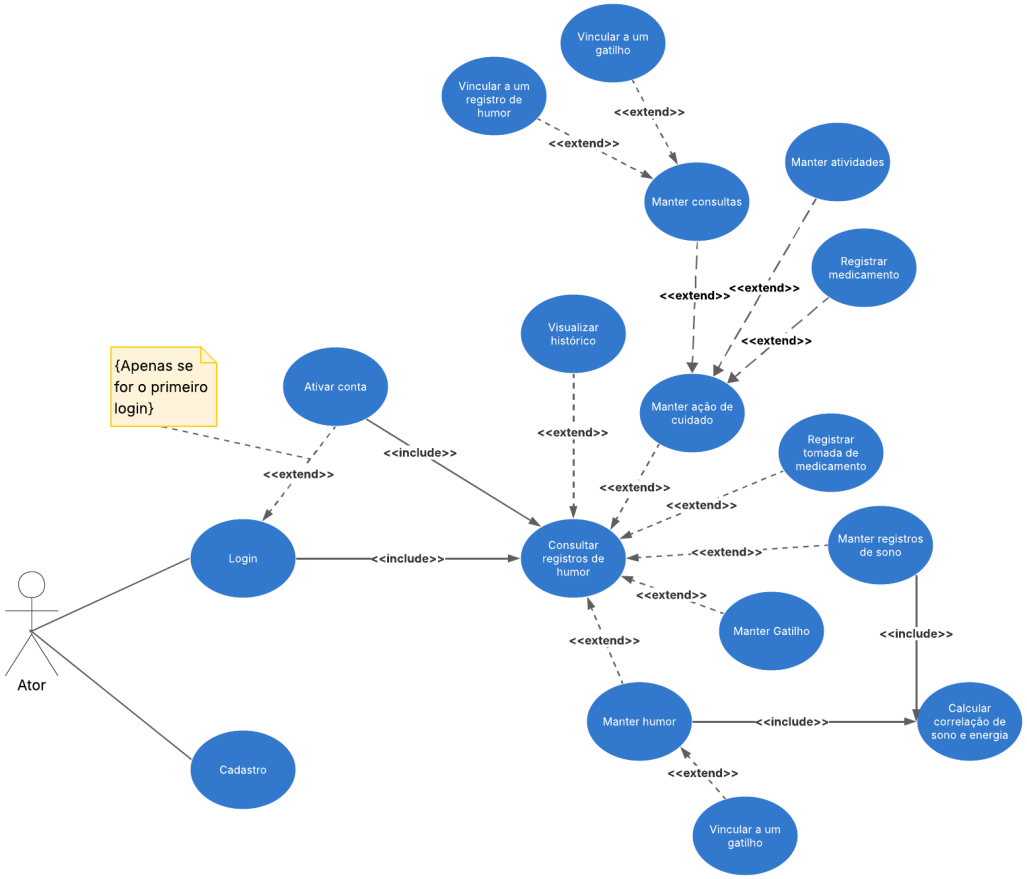
O diagrama de casos de uso, apresenta de forma simples, os atores, que são representações das entidades que interagem com o sistema (sejam eles usuários finais ou outros sistemas), e os casos de uso, que são as funcionalidades que o sistema oferece. Ambos são conectados por linhas de associação. O sistema também é delimitado por um retângulo, que contém os casos de uso (IBM Documentation, 2021).

Ao final desse processo, essa modelagem é transformada em documentação, e no Apêndice A deste documento, contém a mesma, oferecendo uma visão mais detalhada para cada caso de uso, seus caminhos alternativos e caminhos de exceções. Pode-se verificar que esse acaba por ser o diagrama mais basilar do projeto inteiro (GUEDES, 2022).

Na Figura 7, pode-se observar que apenas um ator interage com o sistema, o próprio usuário final. Também visualiza-se que cada elipse, se resume a um requisito funcional, e os casos de uso estão interligados por setas nomeadas com a palavra *"«extend»"* que indica conexões onde a execução de um caso de uso adjacente pode ocorrer dado uma determinada condição em adição ao caso de uso inicial (IBM Documentation, 2021).

Permeando todas as interações no sistema, o caso de uso mais central, o "Lo-

Figura 7: Diagrama de Casos de Uso



Fonte: Elaborado pelo autor (2026)

gin", cujo qual estabelece uma base de autenticação, de forma que garante que apenas usuários autenticados possam interagir com o sistema.

No caso de uso "Manter Humor" representa a funcionalidade central do sistema, permitindo que o usuário registre seu estado emocional ao longo do tempo. Essa ação ocorre por meio da inserção direta de uma informação de humor, cujo qual o usuário pode selecionar entre cinco humores "basais", além de elementos que ajudem a compreender, como escalas indicativas dos respectivos níveis de estresse, ansiedade e quais componentes do humor o usuário identifica com mais presença. A manutenção do humor no sistema contempla tanto a criação quanto a edição e a exclusão desses registros.

O caso de uso "Registrar Intervenção" contempla o cadastro de estratégias, práticas ou sugestões que o usuário utiliza ou deseja utilizar para lidar com oscilações emocionais. As ações de cuidados, podem incluir desde ações práticas, como caminhar, até técnicas cognitivas ou expressivas, como atividades criativas, medicamentos utilizados ou atendimentos com profissionais da área. Essa funcionalidade contribui

para que o sistema sirva como uma central e colabora para construir um registro mais criterioso, visto que o usuário reconstrói o que aconteceu naquele determinado ponto no tempo.

O caso de uso “Registrar Gatilho” visa permitir que o usuário registre eventos, sejam eles pensamentos, situações, ou estímulos que tenham influenciado significativamente seu estado emocional. O objetivo é construir um histórico de causas associadas a variações no humor, de modo a fomentar a identificação de padrões recorrentes ou específicos. Essa funcionalidade é especialmente útil para usuários em contextos terapêuticos ou de autoconhecimento, favorecendo a análise de relações entre circunstâncias externas e reações internas.

O caso de “Registro de sono”, permite o usuário, de forma simples, criar no sistema um registro da quantidade de horas de sono, e adicionar alguma anotação breve. Esse elemento corrobora com a construção de indicadores, tanto da média de sono, quanto também de estimular uma correlação entre sono e os níveis de energia informado em cada humor.

Por fim, o caso de uso “Vincular humor a um gatilho” permite ao usuário estabelecer vínculos entre os gatilhos identificados e os registros de humor, formando um mapeamento de ação e reação, ou causa e reação. Essa associação pode servir tanto para consulta futura quanto para, em um futuro, gerar indicadores. Ao conectar eventos e suas respostas, a funcionalidade fortalece o papel do sistema como uma ferramenta alinhada com o EMA, gerando assim uma narrativa emocional diária.

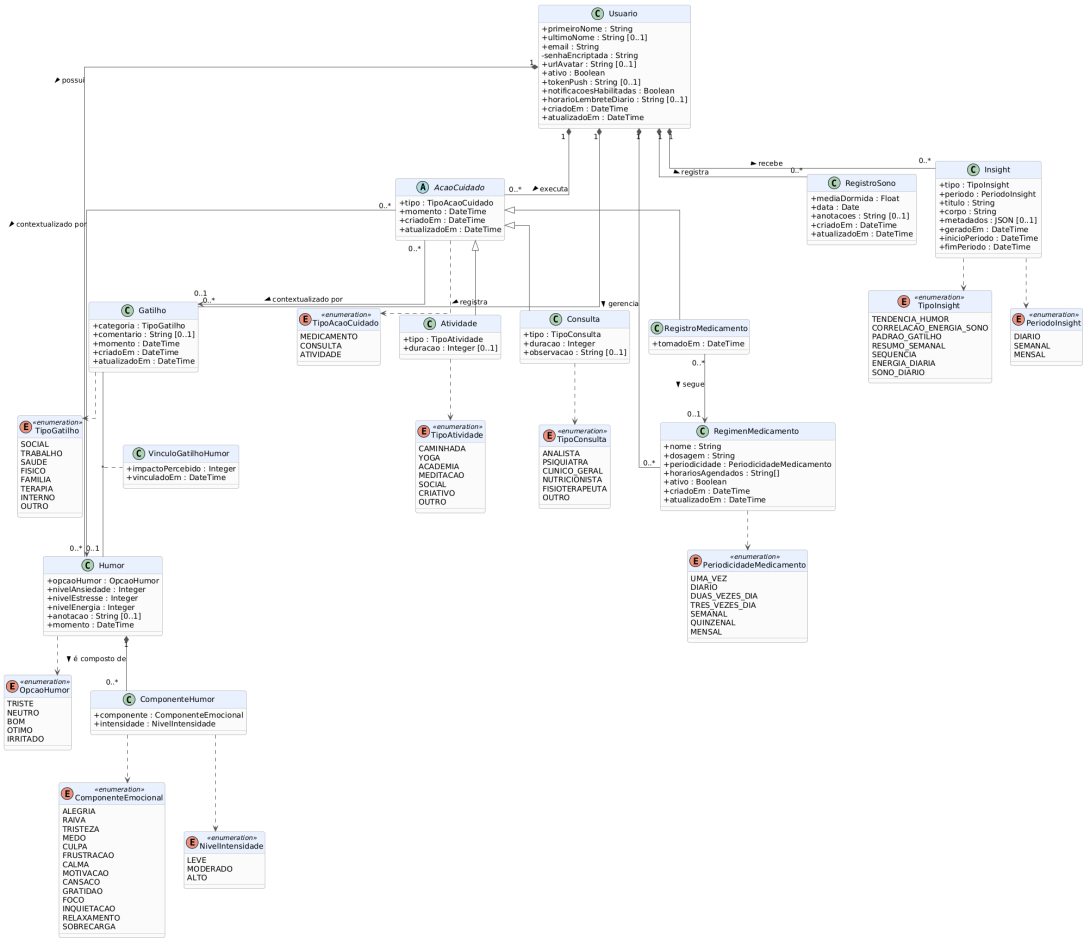
4.2.2 Diagrama de Classes

O diagrama de classes é um dos principais artefatos da UML, responsável por evidenciar a estrutura estática do sistema, seus atributos, operações e relacionamentos. Conforme (GUEDES, 2022, p. 101): “O principal enfoque é permitir a visualização das classes que o sistema e seus respectivos métodos e atributos”. Dessa forma, esse diagrama constitui a base da modelagem estrutural que posteriormente será implementada em código.

Sobre o diagrama em si, cada classe possui três partes: título, os atributos, que representam as características da classe em si, e seus métodos, que são as operações que o sistema pode operar nos objetos das classes (GUEDES, 2022). Na Figura 8, pode-se observar os elementos descritos acima:

Essa representação apresenta alguns vários elementos importantes. Primeiro, um acoplamento entre *Usuário* e as demais classes. Isso é natural, e desejado, considerando que é o único ator. Existe também a separação entre as entidades como Gatilho, Humor, muito embora exista uma relação de associação entre elas, através da classe denominada “VinculoGatilhoHumor”, visto que ela seria representada por uma

Figura 8: Diagrama de Classes



Fonte: Elaborado pelo autor (2026)

tabela auxiliar ligando as duas, aliadas do atributo de estimativa de impacto.

Necessário destacar aqui que a classe “AcaoCuidado” é uma classe abstrata, representando uma generalização. Essa classe serve como um guarda-chuva para representar diversos tipos de ações que o usuário pode realizar. O papel dela aqui se torna guardar elementos em comum entre esses tipos de ações, seus respectivos vínculos com gatilhos ou humores. A partir dela então, traça-se outras sub-classes que contem um melhor contexto sobre a atividade em si, por exemplo, na classe “Consulta” guarda-se dados como a duração da consulta, que tipo de consulta, e potenciais anotações ou comentários em cima da mesma. Entretanto, uma classe “RegistroMedicamento” não precisa guardar mais do que a relação com qual medicamento e quando ele foi de fato tomado.

Por fim, existe diversos tipos de enumeradores que delimitam exatamente quais tipos de valores podem ser escolhidos. Esses enumeradores servem também como um mecanismo de validação, sendo movidos tanto para o próprio banco de dados e posteriormente sendo implementados como forma de controle em outros pontos como na API.

Ressalta-se, contudo, que o diagrama não contempla, padrões arquiteturais complementares, como *Service Layer*, *Repository* e *Data Access Object (DAO)*, os quais são incorporados na etapa de implementação para garantir uma separação adequada de responsabilidades e facilitar a manutenção e escalabilidade do código, conforme as propostas de FOWLER (2013) e GAMMA et al. (2011). Esses elementos são importantes, considerando os princípios de legibilidade, simplicidade e responsabilidades, muitos organizados por MARTIN (2012), asseguram que o código produzido seja limpo e sustentável.

5 DESIGN

As decisões de design do sistema foram fundamentadas tanto na análise dos aplicativos similares quanto nos resultados do questionário aplicado durante a etapa de pesquisa. Esses elementos permitiram identificar padrões recorrentes de interface e compreender as preferências do público-alvo, orientando a definição da identidade visual, da linguagem adotada e da organização dos elementos da interface.

A análise dos aplicativos similares revelou uma linguagem visual comum entre as soluções existentes, caracterizada por interfaces lúdicas, linguagem informal e, em alguns casos, uso intenso de cores. Paralelamente, os resultados do questionário indicaram que 37,5% dos participantes preferem uma comunicação acolhedora e empática, enquanto 25% demonstraram preferência por uma linguagem neutra e objetiva.

O design, no contexto de desenvolvimento de software é um elemento fundamental, considerando que influencia diretamente a experiência do usuário, na percepção e assimilação das informações.

Diferentemente da maioria dos aplicativos analisados, optou-se por uma identidade visual menos lúdica e mais sóbria, reduzindo o uso de ilustrações e cores saturadas em favor de uma linguagem mais discreta e orientada ao conteúdo.

5.1 IDENTIDADE VISUAL

Identidade visual, neste trabalho, designa o conjunto de decisões gráficas que conferem ao aplicativo uma aparência reconhecível e consistente: o nome e o logotipo, a hierarquia visual, a paleta de cores e a tipografia. Esses elementos constituem a primeira camada de comunicação entre o sistema e o usuário e materializam as preferências identificadas na etapa de pesquisa — em especial a opção por uma linguagem sóbria e acolhedora. Como apontam os princípios universais de design, a consistência e o reconhecimento desses elementos reduzem a carga de interpretação e tornam a interface mais previsível (LIDWELL; HOLDEN; BUTLER, 2011). As subseções seguintes percorrem cada uma dessas decisões.

5.1.1 Nome e Logotipo

O termo Nexo remete à ideia de conexão, relação e convergência entre diferentes elementos. Esse conceito dialoga diretamente com a proposta do projeto, fundamentada na metodologia de Avaliação Ecológica Momentânea (EMA). Embora seja cada vez mais comum monitorar aspectos isolados da rotina, como qualidade do sono, nível de atividade física ou uso de medicamentos, essas informações frequentemente permanecem fragmentadas, dificultando a compreensão das relações existentes entre elas.

Nesse contexto, o nome Nexo representa a proposta central do sistema: integrar essas informações e auxiliar o usuário na construção de uma narrativa coerente sobre seu estado emocional. Em vez de tratar humor, emoções, estresse, energia, ansiedade, atividades, psicofármacos e consultas como registros independentes, o aplicativo busca evidenciar como esses fatores se relacionam ao longo do tempo, favorecendo a identificação de padrões e possíveis associações.

Figura 9: Logotipo do sistema



Fonte: Elaborado pelo autor (2026)

Assim, tanto o nome quanto o logotipo procuram transmitir a ideia de convergência, de “dar a liga”, representando o papel do sistema em conectar informações dispersas e transformá-las em conhecimento útil para o processo de autorreflexão.

Para convergir com a pesquisa de tonalidade, e abraçar uma estética “acolhedora”, o logo tenta fugir de parecer algo mecânico ao usar uma técnica que simule pinceladas ao invés de linhas retas, simulando também diferenças ou falhas na pressão utilizada por ter pequenas variações e espaços em branco no logotipo.

5.1.2 Hierarquia visual

No contexto de interfaces digitais, a hierarquia visual corresponde à estruturação de elementos de uma tela ou página de forma a narrar e orientar o usuário com os elementos que são mais significativos e que demandam mais atenção.

Por meio de repetições de padrões de agrupamento de informações, contraste, tamanhos, espaçamentos, busca-se reduzir o esforço necessário para encontrar elementos importantes e quais ações necessitam de maior visibilidade. Segundo LIDWELL; HOLDEN; BUTLER (2011) uma hierarquia visual bem definida, ajuda o usuário a interpretar rápidamente a interface, reduzindo a carga cognitiva e tornando a navegação mais intuitiva.

Portanto, com esse foco em mente, produziu-se uma lista pequena de componentes com visual semelhante, reduzindo o contraste entre esses mesmo elementos, o que ajuda justamente a gerar uma sensação de consistência e previsibilidade entre telas e elementos gráficos.

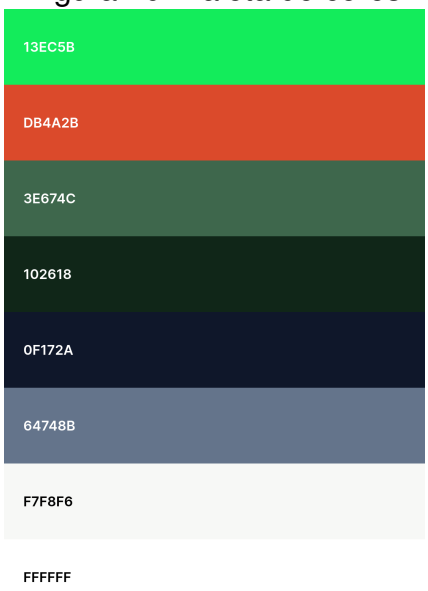
A estrutura da aplicação pode ser resumida em cinco componentes fundamentais: cartões (*cards*), botões de ação, *chips* para categorização de informações, campos de formulário e indicadores em escala (*sliders*). Esses componentes aparecem de maneira recorrente em praticamente todas as telas do sistema, permitindo que o usuário reconheça rapidamente sua função sem necessidade de reaprendizado.

Portanto, a organização de telas geralmente se dá nesses mesmos parâmetros, em que toda tela é dividida em seções, compostas de um cabeçalho, seguido dos respectivos elementos de interação. Posteriormente, a utilização de cartões, ajuda a reforçar esse foco visual, agrupando elementos que são interativos e/ou que necessitam interação e/ou atenção.

5.1.3 Paleta de Cores

Como discutido anteriormente, a definição da paleta de cores buscou equilibrar dois objetivos identificados durante a fase de pesquisa, transmitir uma identidade visual acolhedora, porém evitando excessos de cores e informações lúdicas. Portanto, para buscar esses pilares, optou-se por usar uma escolha de cores mais neutras e sóbrias, focando em tons de branco, cinza e preto, de forma a permitir que o conteúdo regstrado pelo usuário seja o maior foco de destaque.

Figura 10: Paleta de cores



Fonte: Elaborado pelo autor (2026)

Contudo, para elementos que necessitam de uma maior atenção, ou elementos de interação, como botões de ações primárias, adotou-se uma escolha mais rádical, um tom de verde forte, que busca criar exatamente a sensação de destaque imediato. Além disso, o verde geralmente está associado como uma cor que transmite uma sensação de vitalidade, força, natureza, bem-estar e afins. Como mencionado anteriormente, limitou-se a utilização para indicadores, componentes de interação e elementos que representam progresso ou confirmação.

Por fim, foi escolhido outras cores pasteis para ajudar a destacar elementos como sentimento e emoções, indicadores para cada tipo de registro gerado no sistema, de forma mínima e não intrusiva.

5.1.4 Tipografia

A tipografia constitui um dos principais elementos responsáveis pela legibilidade em interfaces digitais. Segundo o Material Design (GOOGLE, 2021), a escolha adequada da fonte influencia diretamente a velocidade de leitura, a organização da informação e a percepção geral de qualidade da interface. Em aplicações móveis, onde o espaço disponível é reduzido, a clareza tipográfica torna-se ainda mais importante.

Considerando esses aspectos, optou-se pela utilização da fonte Inter, desenvolvida por Rasmus Andersson especificamente para ambientes digitais. A família tipográfica foi projetada para manter elevada legibilidade em diferentes tamanhos de tela, apresentando boa diferenciação entre caracteres e excelente renderização em sistemas operacionais modernos (ANDERSSON, 2016).

A hierarquia tipográfica do sistema foi construída a partir de poucas variações

Figura 11: Referência da fonte Inter



Fonte: Site da página da fonte Inter (2026)

de tamanho e peso da fonte. Títulos utilizam maior peso tipográfico para estabelecer prioridade visual, enquanto textos informativos e descrições empregam pesos mais leves, mantendo consistência em toda a aplicação. Essa padronização reduz a quantidade de estímulos visuais simultâneos e facilita a identificação das informações mais relevantes em cada tela.

Além da legibilidade, a escolha da Inter também contribui para reforçar a identidade moderna e funcional proposta pelo aplicativo. Por ser amplamente utilizada em plataformas digitais e ferramentas de interface, como o Figma, a fonte transmite familiaridade ao usuário e mantém consistência com os padrões contemporâneos de design de interfaces.

5.2 ESTRUTURA

Por estrutura entende-se a organização das telas e da navegação do aplicativo: o que cada tela contém, em que ordem os elementos se apresentam e como o usuário se desloca entre elas. No Nexo, essa organização repousa sobre dois alicerces: uma navegação por abas que divide o sistema em cinco áreas — Histórico, Jornada, Novo, Ações de Cuidado e Configurações — e a composição das telas por cartões reutilizáveis, descritos na seção anterior. Esta seção apresenta como essa estrutura foi planejada e mapeada antes da aplicação da identidade visual definitiva.

5.2.1 Organização da navegação

A navegação do Nexo é organizada em cinco áreas, acessíveis por uma barra de abas inferior: “Histórico”, “Jornada”, “Novo”, “Ações de Cuidado” e “Configurações”. A divisão separa as superfícies de consulta — Histórico e Jornada, que apresentam registros e indicadores já consolidados — do ponto de entrada rápido para o registro de humor, a aba Novo. As ações de cuidado, por reunirem modalidades distintas

(medicamentos, consultas e atividades), são agrupadas em um único hub, evitando que cada tipo ocupe uma entrada própria na navegação.

5.2.2 Consolidação de registros no Histórico

Uma decisão estrutural relevante foi consolidar os registros em uma única linha do tempo. Em vez de listas separadas para humor, sono, gatilhos e ações de cuidado, a tela de Histórico combina os quatro tipos em uma sequência cronológica única, agrupada por dia, por meio de um componente de mapeamento que normaliza cada recurso em um formato comum, descrito no Capítulo 7.5. A motivação decorre da própria metodologia EMA: aproximar visualmente eventos relacionados — como um gatilho registrado minutos antes de um humor — permite reconstruir uma narrativa do dia em vez de fragmentar o contexto em telas distintas.

5.2.3 Wireframes

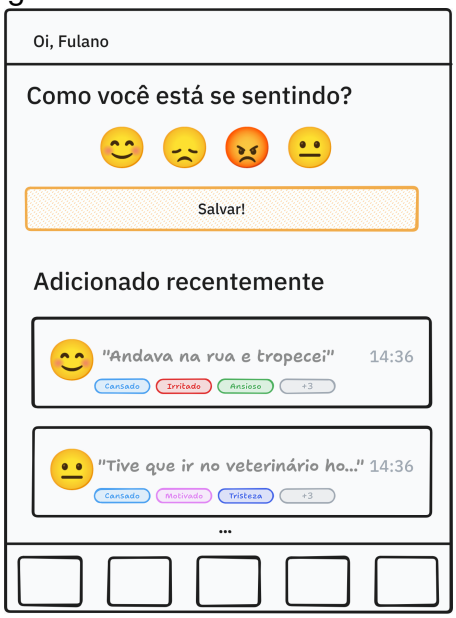
Wireframes são representações simplificadas de uma interface, focadas na estrutura e na disposição dos elementos, sem se preocupar ainda com componentes visuais definitivos, cores ou tipografia. Essa camada de abstração permite válidar rapidamente o que cada tela precisa conter e como o usuário navegaria entre elas, antes de investir tempo no design em alta fidelidade, no qual esses elementos visuais são então aplicados. Quando elementos clicáveis/interativos são adicionados aos *wireframes*, eles também podem se tornar protótipos, usados para modelar e avaliar o fluxo de navegação entre as telas.

No desenvolvimento do aplicativo, essa prática foi usada para mapear as telas existentes e definir quais elementos cada uma apresentaria, como exemplifica a tela inicial na Figura 12:

Alguns elementos podem ser notados nessa tela, como a presença de *cards* e emojis, utilizados para criar destaque visual. A escolha de emojis foi inspirada pelo aplicativo analisado no Capítulo 3, DailyBeans que utiliza ícones para cada tipo de emoção. Portanto, a escolha de emojis se dá como uma saída rápida e fácil, visto que não demanda conhecimentos de íconografia.

Um segundo *wireframe*, apresentado na Figura 13, mostra uma proposta inicial para a tela de Indicadores. Nessa versão inicial, o usuário selecionava, por meio de um menu suspenso, (“Flutuação de:”), qual indicador desejava visualizar, sendo os dados apresentados em um gráfico de linha que representava a flutuação dos níveis informados de um humor (“Energia”, “Ansiedade” e/ou “Estresse”), além de seções para gatilhos frequentes e um indicador de estabilidade de humor.

Figura 12: *Wireframe* da tela inicial



Fonte: Elaborado pelo autor (2026)

Figura 13: *Wireframe* da tela de indicadores



Fonte: Elaborado pelo autor (2026)

Essa abordagem, contudo, foi descartada na versão final do aplicativo: o gráfico com seletor de tipo de indicador foi removido, e a tela foi repaginada para simplificar usando três tipos de indicadores automaticamente cálculados pelo sistema, conforme detalhado no Capítulo 7.4.

6 TECNOLOGIAS

Este capítulo apresenta as principais tecnologias empregadas no desenvolvimento do aplicativo, organizadas de acordo com a camada da arquitetura em que atuam: *front-end*, *back-end* e infraestrutura/persistência de dados.

6.1 FRONT-END

O termo *frontend* refere-se à camada de um sistema responsável pela interação direta com o usuário: a interface visual, a lógica de apresentação e o processamento que ocorre no dispositivo do próprio usuário, seja um navegador ou, como no caso do Nexo, um aplicativo móvel, conforme SOMMERVILLE (2010). É o frontend que renderiza telas, captura entradas do usuário (como toques e formulários) e comunica-se com o *backend* através de requisições de rede, geralmente utilizando APIs. No caso do Nexo, essa camada é implementada com *React Native* e Expo, permitindo que a mesma base de código seja compilada tanto para Android utilizando bibliotecas em *TypeScript*.

6.1.1 React Native

React Native é um *framework* de código aberto, mantido pela Meta, que permite o desenvolvimento de aplicativos móveis multiplataforma a partir de uma única base de código escrita em JavaScript/TypeScript, utilizando componentes que são traduzidos para elementos nativos tanto no iOS quanto no Android (Meta Platforms, Inc., 2026).

A escolha do React Native se justifica pelo escopo reduzido da equipe de desenvolvimento (um único desenvolvedor) e facilidade de conhecimento e manutenção, sem trazer, para este projeto, benefícios que justifiquem esse custo adicional. Além disso, o ecossistema maduro do React Native oferece ampla disponibilidade de bibliotecas para funcionalidades já previstas no aplicativo, como notificações locais e gráficos.

6.1.2 Zustand

Zustand é uma biblioteca leve de gerenciamento de estado para aplicações React e React Native, que dispensa a estrutura verbosa de *providers* e *reducers* tipi-

camente associada a bibliotecas como Redux (Poimandres, 2026).

Ela foi escolhida para gerenciar o estado efêmero da aplicação (por exemplo, dados de formulários em preenchimento ou estados de navegação temporários) por sua API minimalista e baixo *overhead* de configuração, adequada a um projeto de escopo individual em que a complexidade adicional de soluções mais robustas de gerenciamento de estado não se justifica.

6.2 BACK-END

Já o termo *backend* refere-se à camada de um sistema que processa a lógica de negócio, gerencia o acesso e a persistência de dados, e expõe essa funcionalidade a clientes (como o frontend) através de uma interface bem definida, tipicamente uma API HTTP (FOWLER, 2013). O *backend* é responsável por validar dados recebidos, aplicar regras de negócio, autenticar usuários, e coordenar o acesso ao banco de dados e a outros serviços auxiliares. No caso do Nexo, essa camada é implementada em Node.js com o framework *Express*, seguindo uma arquitetura em camadas (*Route* $\rightarrow$ *Controller* $\rightarrow$ *Service* $\rightarrow$ Prisma) que separa as responsabilidades de validação, lógica de negócio e acesso a dados.

6.2.1 Express.js

Express.js é um *framework* minimalista para aplicações web em Node.js, amplamente utilizado para a construção de APIs *REST*, oferecendo uma camada fina de abstração sobre o roteamento de requisições HTTP e *middlewares* (OpenJS Foundation, 2026).

Sua adoção se deve à simplicidade e à flexibilidade não opinativa do *framework*, que permite estruturar a API de acordo com as necessidades específicas do projeto, sem impor convenções rígidas de arquitetura, além de sua ampla documentação e maturidade no ecossistema Node.js.

6.2.2 Zod

Zod é uma biblioteca de validação e definição de esquemas de dados para TypeScript, que permite declarar formatos esperados de entrada (como corpos de requisição) e inferir automaticamente os tipos estáticos correspondentes (Colin McDonnell, 2026).

Zod foi escolhido para validar os dados recebidos pela API antes de seu processamento, garantindo que registros de humor, sono e demais entradas do usuário estejam em conformidade com o formato esperado. A inferência automática de tipos a partir dos esquemas evita a duplicação de definições de tipo entre a camada de

validação e o restante do código TypeScript do back-end.

Código 1: Validação e Inferência usando Zod

```
1  import {
2    BaseMoodOption,
3    MoodComponentOption,
4    IntensityLevel
5  } from '@prisma/client';
6
7  export const MoodComponentSchema = z.object({
8    component: z.nativeEnum(MoodComponentOption),
9    intensity: z.nativeEnum(IntensityLevel)
10 });
11
12 export const CreateMoodSchema = z.object({
13   annotation: z.string().optional(),
14   moment: z.coerce.date(),
15   selectedMood: z.nativeEnum(BaseMoodOption),
16   anxietyLevel: z.number().int().min(0).max(10),
17   stressLevel: z.number().int().min(0).max(10),
18   energyLevel: z.number().int().min(0).max(10),
19   moodComponents: z.array(MoodComponentSchema).min(1)
20 });
21
22 export type CreateMoodInput = z.infer<typeof CreateMoodSchema>;
```

Fonte: Elaborado pelo autor (2026)

Aqui, é possível observar alguns elementos, como a extração direta dos *enums* mapeados pelo Prisma, citado anteriormente nas linhas 1–5. Em seguida, são criados os *schemas* de validação: um valida os componentes de humor de forma aninhada dentro do objeto recebido, e o outro valida os dados do próprio registro — os níveis aceitos de ansiedade, estresse e energia; a conversão do campo `moment` para um objeto de data a partir da entrada recebida; a exigência de ao menos um componente de humor selecionado; entre outros. Por fim, esses dados são exportados como um *type* do TypeScript, usado pelo compilador para a verificação estática de tipos no restante do back-end.

6.2.3 Valkey e BullMQ

Valkey é um banco de dados em memória, do tipo chave-valor, criado como *fork* de código aberto do Redis, mantido sob a governança da Linux Foundation (Valkey Contributors, Linux Foundation, 2026). BullMQ é uma biblioteca para gerenciamento de filas de tarefas em Node.js, construída sobre bancos de dados compatíveis com o protocolo Redis (Taskforce.sh Inc., 2026).

A combinação das duas ferramentas foi adotada para o processamento paralelo e assíncrono de tarefas que não precisam bloquear a resposta imediata da API, como o disparo de notificações programadas e o processamento de indicadores a partir do histórico de registros do usuário. A escolha do Valkey em específico, e não do Redis diretamente, se deve à mudança de licenciamento do Redis em 2024, que passou a restringir certos usos comerciais; o Valkey mantém compatibilidade total de protocolo, permanecendo sob licença permissiva de código aberto.

6.3 INFRAESTRUTURA E PERSISTÊNCIA DE DADOS

A camada de *infraestrutura e persistência* de um sistema abrange os componentes responsáveis por armazenar e gerenciar os dados de forma durável, bem como os serviços auxiliares que sustentam o funcionamento da aplicação em produção (FOWLER, 2013). O elemento central dessa camada costuma ser o sistema de gerenciamento de banco de dados (SGBD), responsável por garantir a integridade, consistência e durabilidade dos dados armazenados. No caso do Nexo, essa função é cumprida pelo PostgreSQL, um SGBD relacional, acessado através do Prisma, um *Object-Relational Mapper* (ORM) que traduz operações do banco de dados em chamadas de código TypeScript, eliminando a necessidade de escrever consultas SQL manualmente na maior parte dos casos.

Complementarmente, o Nexo utiliza o Valkey como armazenamento em memória de alta velocidade, empregado tanto para cache quanto como *broker* de mensagens para o sistema de filas, cujo suas filas são gerenciadas com a biblioteca BullMQ para lidar com os *background jobs* enviados. Os mesmos são responsáveis por processar os indicadores semanais e diários descritos nos capítulos anteriores.

6.3.1 PostgreSQL

PostgreSQL é um sistema gerenciador de banco de dados relacional, de código aberto, reconhecido por sua conformidade com o padrão SQL, robustez e suporte a tipos de dados avançados (PostgreSQL Global Development Group, 2026).

Sua escolha se justifica pela natureza estruturada e relacional dos dados do domínio da aplicação (registros de humor associados a usuários, horários, gatilhos e categorias), que se beneficia das garantias de integridade referencial e das transações ACID oferecidas por um banco relacional maduro, em detrimento de soluções não relacionais.

6.3.2 Prisma

Prisma é um *Object-Relational Mapper* (ORM) para o ecossistema Node.js/TypeScript, que gera automaticamente um cliente de acesso ao banco de dados tipado a partir de um esquema declarativo, além de oferecer um sistema de migrações versionadas (Prisma Data, Inc., 2026).

O Prisma foi escolhido por permitir a manutenção do esquema do banco de dados de forma versionada e sincronizada com o código da aplicação, reduzindo a chance de divergência entre a estrutura do banco e as entidades manipuladas pelo back-end, além de prover checagem de tipos em tempo de compilação para as consultas realizadas.

6.4 IMPLANTAÇÃO E DESENVOLVIMENTO CONTÍNUO

Já os termos *implantação* (*deployment*) e *automatização* referem-se ao conjunto de práticas e ferramentas utilizadas para transformar o código-fonte de uma aplicação em um sistema efetivamente disponível e em execução, de forma repetível e com o mínimo de intervenção manual (HUMBLE; FARLEY, 2011). No caso do Nexo, essa camada é sustentada por três ferramentas complementares. O Docker é utilizado para empacotar a aplicação e suas dependências em contêineres isolados e portáveis, garantindo que o ambiente de execução seja idêntico entre desenvolvimento e produção; o Docker Compose orquestra a execução conjunta de múltiplos contêineres (aplicação, banco de dados, Valkey), definindo suas configurações e interdependências em um único arquivo declarativo. O GitHub Actions é responsável pela integração e entrega contínuas (CI/CD), automatizando etapas como execução de testes e construção de imagens a cada alteração no código. Por fim, o Ansible é utilizado para o provisionamento inicial da infraestrutura, automatizando a configuração do servidor onde a aplicação é hospedada.

6.4.1 Docker e Docker Compose

Docker é uma plataforma de conteinerização que permite empacotar uma aplicação e suas dependências em uma unidade isolada e portátil, capaz de ser executada de forma consistente em diferentes ambientes (Docker, Inc., 2026a). O Docker Compose é uma ferramenta complementar que permite definir e orquestrar múltiplos contêineres — e as relações entre eles — por meio de um único arquivo de configuração declarativo (Docker, Inc., 2026b).

No projeto, o Docker Compose é utilizado tanto no ambiente de desenvolvimento quanto no ambiente de produção para orquestrar os serviços de banco de dados (PostgreSQL), armazenamento em memória (Valkey) e da própria API, garantindo

que as versões e configurações desses serviços sejam idênticas entre as máquinas de desenvolvimento e o servidor de produção.

O uso de *healthchecks* em conjunto com a diretiva `depends_on` e a condição `service_healthy` assegura que a API só seja iniciada após o banco de dados e o Valkey estarem de fato prontos para aceitar conexões, evitando falhas de inicialização por condição de corrida (*race condition*), como ilustrado no Código 2.

Código 2: Configuração do *healthcheck* do Valkey e de dependência condicional da API no `docker-compose.yml`

```yaml
valkey:
  image: valkey/valkey:9.0-alpine
  healthcheck:
    test: ["CMD", "valkey-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 5s

api:
  build: ./api
  depends_on:
    valkey:
      condition: service_healthy
    db:
      condition: service_healthy
```

Fonte: Elaborado pelo autor (2026)

A adoção do Docker se justifica, no contexto de uma aplicação de saúde mental de código aberto, pelo objetivo de garantir a reprodutibilidade do ambiente de execução: qualquer pessoa que clone o repositório — seja para contribuir com o projeto, auditar seu funcionamento ou realizar sua própria implantação — consegue reproduzir o ambiente completo sem depender de configurações manuais específicas do sistema operacional hospedeiro, o que reforça o caráter *self-service* do projeto.

6.4.2 GitHub Actions

GitHub Actions é uma plataforma de integração e entrega contínuas (CI/CD) nativa do GitHub, que permite a definição de fluxos de trabalho automatizados — disparados por eventos como *push* ou *pull request* — descritos de forma declarativa em arquivos YAML (GitHub, Inc., 2026).

No projeto, o fluxo de trabalho de CI/CD é dividido em três etapas sequenciais e dependentes entre si. A primeira etapa (`test`) sobe um serviço efêmero de PostgreSQL e executa a suíte de testes automatizados da API a cada *push* ou *pull request*

que modifique arquivos do diretório da API.

A segunda etapa (`build-and-push`), condicionada ao sucesso da primeira e restrita a eventos de *push*, constrói a imagem Docker de produção da API e a publica no GitHub Container Registry (GHCR), com marcação automática pelo SHA do *commit* e, condicionalmente, pela tag `latest` quando o *push* ocorre na *branch* `master`.

Por último, a terceira etapa (`deploy-production`), restrita à *branch* `master`, conecta-se ao servidor de produção via SSH e realiza a atualização do serviço da API por meio da atualização da tag da imagem no arquivo `.env` e da recriação do contêiner correspondente via Docker Compose.

Esse processo elimina a necessidade de intervenção manual para a maior parte do ciclo de implantação, restringindo a ação humana direta sobre o servidor de produção a situações excepcionais, o que reduz a superfície de erro humano e aumenta a confiabilidade do processo de entrega de novas versões da aplicação.

6.4.3 Ansible

Ansible é uma ferramenta de automação de infraestrutura que permite descrever, de forma declarativa e idempotente, o estado desejado de um ou mais servidores, dispensando a necessidade de um agente instalado na máquina gerenciada, uma vez que se comunica com ela via SSH (Red Hat, Inc., 2026).

No escopo deste projeto, o papel do Ansible é ortogonal ao do Docker Compose e ao do GitHub Actions: enquanto estes automatizam, respectivamente, a orquestração dos serviços da aplicação e o ciclo de integração e entrega contínuas, o Ansible é responsável exclusivamente pelo provisionamento inicial do servidor de produção, executado uma única vez — ou sempre que um novo servidor precisar ser provisionado. Esse provisionamento consiste em: (1) criar o usuário dedicado à aplicação; (2) adicionar a chave pública SSH utilizada pelo GitHub Actions para se autenticar no servidor durante a etapa de implantação; (3) criar os diretórios da aplicação com as permissões adequadas; e (4) instalar o Docker no servidor. A cópia do arquivo de variáveis de ambiente de produção (`.env.production`), por conter segredos e credenciais sensíveis, é deliberadamente mantida como uma etapa manual, não automatizada, e não é versionada no repositório.

A separação entre o provisionamento inicial (*Ansible*), a orquestração de execução (*Docker Compose*) e a automação do ciclo de entrega (*GitHub Actions*) reflete uma decisão arquitetural: cada ferramenta atua em uma camada distinta e bem delimitada do processo de implantação, o que favorece a reprodutibilidade de todo o processo, do zero, por qualquer pessoa que deseje hospedar sua própria instância da aplicação.

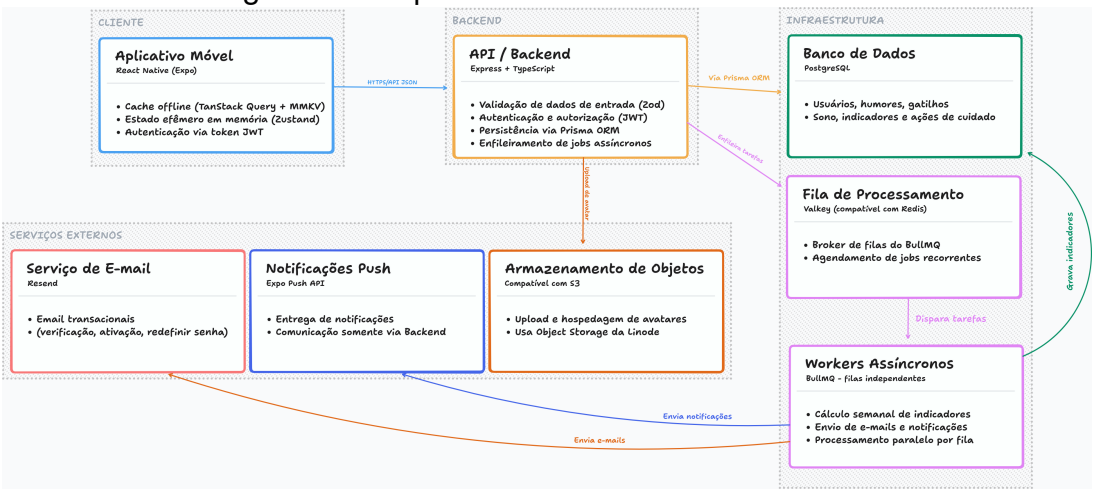
Essa preocupação é particularmente relevante no contexto de um aplicativo de

saúde mental de código aberto: ao reduzir a dependência de serviços proprietários de implantação (*walled gardens*) e ao documentar e automatizar o processo de provisionamento, o projeto favorece um modelo mais alinhado aos princípios de privacidade e de autonomia sobre os dados dos usuários, permitindo que instituições ou indivíduos preocupados com a confidencialidade de dados sensíveis de saúde mental hospedem e controlem sua própria instância da aplicação.

7 ARQUITETURA E FUNCIONAMENTO DO SISTEMA

Como delineado as tecnologias no capítulo anterior, aqui se apresenta a arquitetura, demonstrando como todas as tecnologias apresentadas acima se conectam e conversam entre si.

Figura 14: Arquitetura e elementos do Sistema



Fonte: Elaborado pelo autor (2026)

Este capítulo detalha o funcionamento de algumas funcionalidades desenvolvidas durante o projeto. Apenas algumas funcionalidades mais chaves são abordadas neste capítulo, como o registro de humor, o vínculo de um gatilho, os indicadores, o registro de atividades e, por fim, a exportação de dados e a geração de relatórios.

7.1 PROCESSO DE LOGIN

Logo ao abrir o aplicativo, o usuário é redirecionado para a tela de login. A tela em si é relativamente simples, e contém os campos de email e senha. Existe aqui um elemento proposital: uma forma de garantir um certo nível de anonimato, em que o usuário não precisa se identificar com um nome ou número de telefone, tampouco ativar a conta através de uma plataforma de terceiro, como Google ou Facebook.

Isso se dá, em especial, por dois motivos. Primeiro, o uso de uma tecnologia de autenticação, como por exemplo, *OAuth* (HARDT, 2012) exigiria uma conexão com plataformas de terceiro, que poderiam então rastrear quais usuários têm acesso a qual

aplicativo — algo que é fartamente documentado na literatura sobre os mecanismos usados por tais plataformas para rastrear e monitorar o acesso a dados de usuários, com o fim de vender e monetizar essas informações.

Essa preocupação não é hipotética: um estudo que interceptou o tráfego de rede de 36 dos aplicativos mais populares de saúde mental (depressão e cessação de tabagismo) encontrou transmissão de dados a pelo menos um terceiro em 92% deles, sendo que 81% enviavam dados especificamente a serviços operados por Google ou Facebook — e apenas 59% desses casos declaravam esse compartilhamento em política de privacidade (HUCKVALE; TOROUS; LARSEN, 2019).

Um estudo posterior, focado exclusivamente em 27 aplicativos de saúde mental mais populares da Google Play Store, confirmou o mesmo padrão e o caracterizou formalmente como uma ameaça:

> A maioria dos aplicativos representa ameaças relacionadas à rastreabilidade, à identificação e à detecção. Isso constitui um risco, pois alguns terceiros podem rastrear, reidentificar e detectar as ações e os dados dos usuários. A falta de conscientização também está relacionada a essas ameaças, uma vez que os aplicativos não explicam (por exemplo, na política de privacidade) os riscos que a publicidade direcionada representa para pessoas com problemas mentais, bem como o risco de reidentificação e divulgação de condições de saúde mental (por exemplo, ansiedade, depressão). (IWAYA et al., 2022, p. 3, tradução nossa)

Como os próprios autores resumem, a mera vinculação de um usuário a um aplicativo de saúde mental já pode revelar que essa pessoa possui uma condição psicológica — seja ansiedade, depressão, ou, no caso de um aplicativo como o Nexo, Transtorno Afetivo Bipolar —, tornando usuários desses aplicativos particularmente vulneráveis a esse tipo de exposição (IWAYA et al., 2022).

No caso do Nexo, essa cadeia de inferência é especialmente sensível: a presença do aplicativo na lista de apps instalados de um usuário, associada ao seu identificador publicitário, permitiria a uma plataforma como a Meta inferir uma possível condição de saúde mental do usuário para fins de segmentação de anúncios, sem que este tenha conhecimento ou consentimento explícito disso.

Segundo, e retomando a proposta de oferecer ao usuário a possibilidade de *self-service*, aquele que decidisse manter uma instância própria do aplicativo não precisaria de uma conta em uma plataforma de terceiro, nem se cadastrar nelas.

Figura 15: Tela de login



Fonte: Elaborado pelo autor (2026)

Por fim, destaca-se aqui que, conforme mencionado anteriormente na modelagem de casos de uso (figura 7), ao realizar o login pela primeira vez é enviado um email com um código de verificação, seguindo a lógica dos *magic links*. Isso adiciona uma camada extra de segurança, garantindo que apenas o usuário que solicitou o login possa acessar a conta.

7.2 PÁGINA DE CONSULTA DE REGISTROS

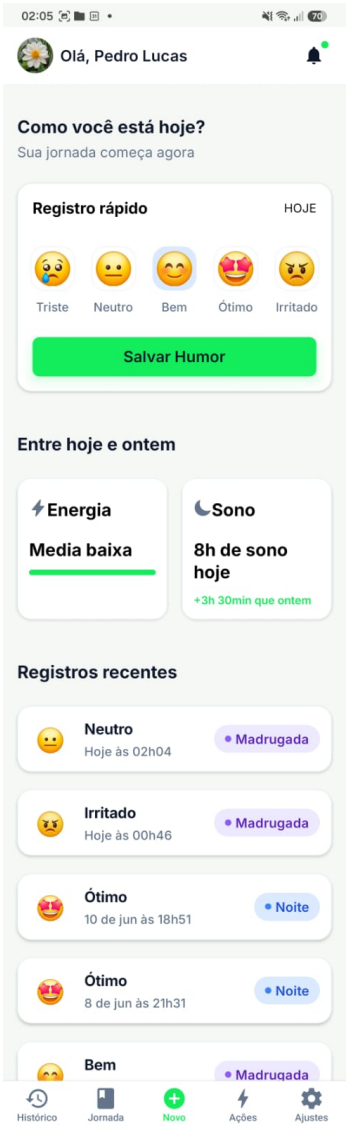
A tela inicial é a tela nomeada apenas como “Novo”. A proposta aqui é ser a tela de cadastro rápido de um humor, e também é a tela que visualmente mais destoa das demais. O usuário é apresentado com três seções distintas:

- Cadastro rápido de um humor

- Entre ontem e hoje
- Registros recentes

Respectivamente: uma seção em que o usuário é convidado a “escolher um humor” e iniciar o processo de registro de uma nova entrada; a seção em que são apresentados os indicadores de nível de energia acumulado entre o dia anterior e o dia atual, além da média de sono do dia comparada com a do dia anterior; e, por fim, a listagem dos dez últimos registros de humor cadastrados pelo usuário.

Figura 16: Tela de Consulta de Registros



Fonte: Elaborado pelo autor (2026)

Aqui já é introduzido ao usuário um padrão de organização e hierarquia: todos os elementos centrais são organizados em *cards*, exibidos em no máximo duas colunas. Também é introduzido, caso o usuário já possua registros de humor cadastrados,

o padrão pelo qual esses registros são resumidos em todas as seções do aplicativo: geralmente um ícone, um título e alguma informação adicional em destaque.

7.3 CRIAÇÃO DE REGISTROS DE HUMOR

Nesta seção, o usuário pode criar novos registros de humor, escolhendo entre uma lista de opções pré-definidas. O humor selecionado na tela anterior é mostrado aqui, e o usuário é convidado a julgar, em uma escala de 1 a 10, como se sente em relação aos níveis de energia, estresse e ansiedade, respectivamente.

Figura 17: Criação de Registros de Humor



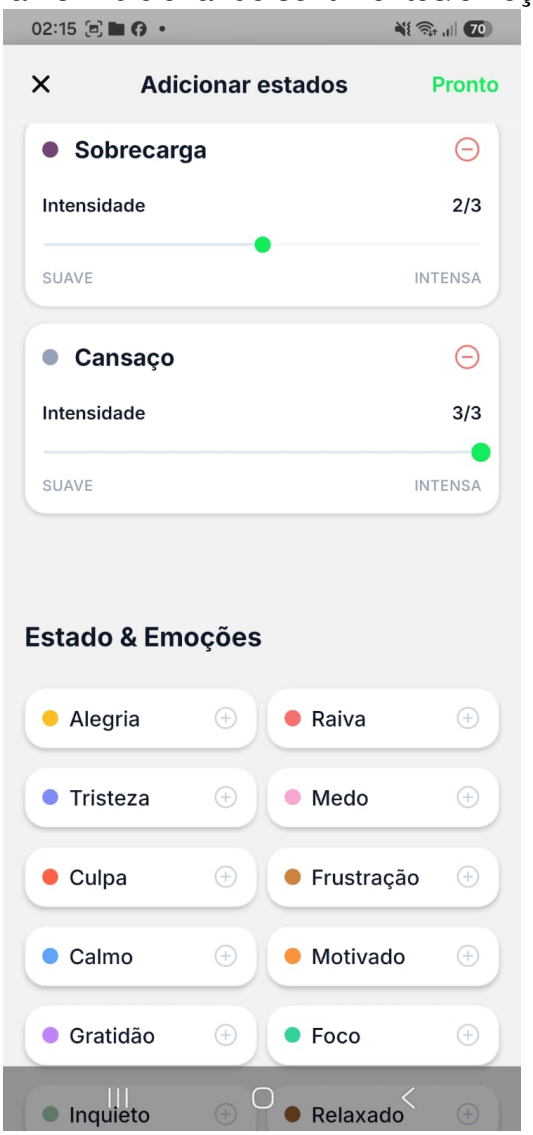
Fonte: Elaborado pelo autor (2026)

Naturalmente, é uma questão difícil de responder — e, como o próprio aplicativo procura deixar claro, é importante que seja um momento reflexivo, não necessariamente preciso.

O usuário pode adicionar uma anotação sobre aquele registro em específico, padrão que se repete em outras telas do aplicativo.

Por fim, o usuário pode escolher o que compõe aquele registro, ou seja, selecionar emoções e estados/sentimentos que sente naquele momento. É apresentada uma listagem mista desses elementos, na qual o usuário seleciona os que mais se aplicam a ele e atribui um nível de intensidade para cada um. Aqui a escala é menor e mais simples, visando facilitar a escolha e o julgamento do usuário, entre “Leve”, “Moderado” e “Intenso”, conforme a figura 18.

Figura 18: Adicionando sentimentos/emoções

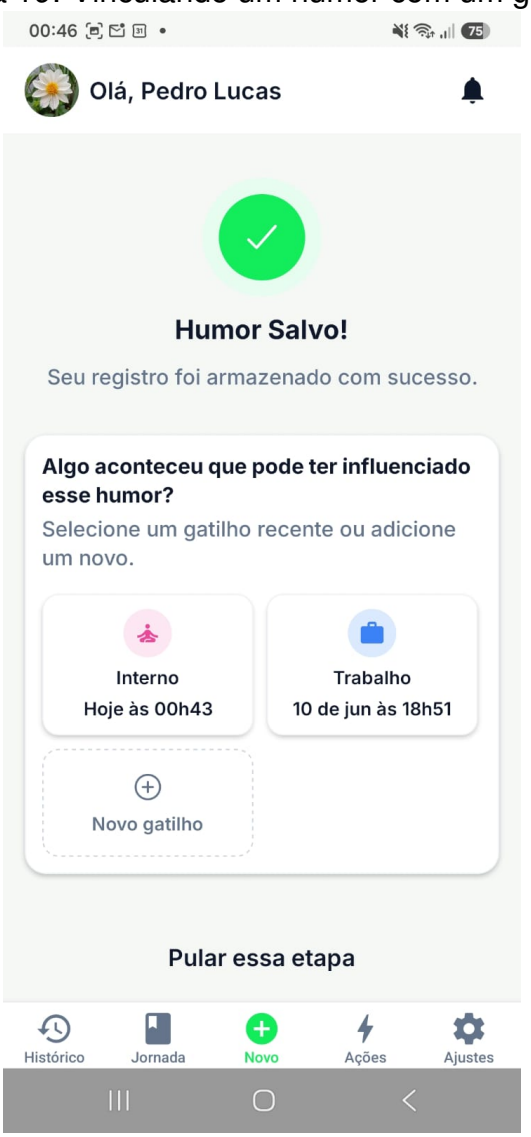


Fonte: Elaborado pelo autor (2026)

Ao concluir esse processo e salvar, é apresentada ao usuário uma tela de confirmação, na qual ele pode, opcionalmente, vincular o registro a um gatilho específico — que pode ter ocorrido antes de ele salvar o registro — ou criar um novo gatilho. O projeto novamente se ancora no EMA, enriquecendo o registro de humor com mais

contexto sobre o que aconteceu.

Figura 19: Vinculando um humor com um gatilho



Fonte: Elaborado pelo autor (2026)

Em seguida, o usuário pode indicar o impacto que aquele gatilho exerceu sobre o registro de humor em questão. Novamente, trata-se de uma estimativa subjetiva, sem pretensão de precisão.

Figura 20: Avaliando o impacto de um gatilho no humor



Fonte: Elaborado pelo autor (2026)

Por fim, vale mencionar que, após esse processo, é enfileirado internamente um *background job* que recalcula, sob demanda, o indicador de energia daquele dia, apresentado na figura 16. Diferentemente deste, os indicadores semanais — como a correlação entre sono e energia, descrita na seção 7.4 — não são recalculados a cada registro individual, sendo processados apenas pelo job semanal agendado (seção 7.4).

7.4 INDICADORES

Nessa tela é apresentado um conjunto de indicadores construídos a partir dos registros de humor, sono e gatilho do usuário. Diferente das demais telas do aplicativo, que operam sobre dados em tempo real, os indicadores são artefatos pré-computados:

um *worker* assíncrono (*BullMQ*) percorre semanalmente todos os usuários ativos e enfileira, para cada um, um conjunto de *jobs* — um por tipo de indicador — que processam a janela dos últimos sete dias e persistem o resultado em uma tabela dedicada. Vale notar que nem todos os indicadores seguem esse fluxo agendado: os indicadores diários (energia e sono) são recalculados sob demanda, no momento em que o usuário registra um novo humor ou sessão de sono, enquanto os indicadores semanais dependem exclusivamente do *job* de *fan-out* de segunda-feira.

O aplicativo móvel apenas consulta esse resultado já calculado, o que evita repetir cálculos custosos a cada abertura da tela e desacopla a lógica analítica do restante da API.

Figura 21: Tela de indicadores



Fonte: Elaborado pelo autor (2026)

São três os indicadores semanais exibidos: tendência de humor, correlação entre sono e energia, e padrão de gatilhos.

- **Tendência de humor**: compara a média de humor da primeira metade da semana com a da segunda metade, expressando a diferença como uma variação percentual. O resultado é classificado como melhora, piora ou estabilidade a partir de uma margem de $\pm 10\%$.
- **Correlação entre sono e energia**: relaciona a duração do sono de uma noite com o nível de energia relatado na manhã seguinte, através do coeficiente de correlação de Pearson. Exige um mínimo de observações pareadas para ser calculado — caso contrário, o indicador simplesmente não é exibido.
- **Padrão de gatilhos**: identifica a categoria de gatilho mais frequente na semana e expressa sua predominância como percentual do total de registros.

É intencional, na figura 21, que o bloco de "Energia" apareça vazio: o cálculo da correlação depende da existência simultânea de registros de sono e de humor em quantidade mínima, e no período retratado o usuário ainda não havia acumulado dados suficientes de ambos os tipos. Optou-se por não exibir uma correlação calculada com poucos pontos, preferindo comunicar a ausência de dado a apresentar um número pouco confiável.

O bloco maior no topo da tela, o "Resumo Semanal", condensa os indicadores mais centrais — média de sono do período e um rótulo de humor ("Estável", no exemplo). Esse rótulo, no entanto, é o elemento mais delicado de toda a funcionalidade. Embora exista uma fórmula capaz de produzir uma classificação de estabilidade a partir da variância dos registros de humor, atribuir esse tipo de afirmação a um usuário — especialmente em um aplicativo que é utilizado por pessoas com Transtorno Afetivo Bipolar, para quem uma leitura equivocada de "estabilidade" pode mascarar justamente o início de um episódio — exige um embasamento mais cuidadoso do que uma regra estatística simples. Por esse motivo, o rótulo apresentado nesta versão do aplicativo é estático (não deriva de fato do cálculo de estabilidade) e a funcionalidade pode ser inteiramente desabilitada pelo usuário nas configurações. A decisão de manter ou não uma afirmação automatizada desse tipo, e sob quais critérios, permanece um ponto em aberto que mereceria revisão de literatura específica antes de ser tratado como funcionalidade definitiva.

7.5 HISTÓRICO

Nessa tela é apresentado o histórico consolidado de registros de humor, sono, gatilho e ações de cuidado (como sessões de terapia), organizados cronologicamente e agrupados por dia, conforme a figura 22. O usuário pode ainda filtrar a listagem por categoria ("Humor", "Sono", "Gatilho" e "Ações de Cuidado"), reduzindo o volume de informação exibida.

Figura 22: Tela de Histórico



Fonte: Elaborado pelo autor (2026)

Como os quatro tipos de dado residem em recursos distintos da API, a tela consulta os respectivos *endpoints* em paralelo e, em seguida, processa o resultado no próprio aplicativo. Para isso, é utilizado um pequeno serviço que atua como *mapper*, transformando os dados de cada *endpoint* em um formato comum, apresentado no código 3.

Código 3: Formato comum da tela de histórico

```
export type HistoryCard = {
  id: string; // "<category>-<id>" to avoid collisions
  category: HistoryCategory;
  subtype?: CareActionSubtype; // only set when category === "care_action"
  timestamp: string; // ISO string -- used for sorting and display
  title: string;
  summary: string;
  badge?: HistoryBadge;
  raw: MoodEntry | SleepRecord | Trigger | CareAction;
};
```

Após a normalização, os quatro conjuntos de dados são combinados em uma única lista ordenada pelo campo `timestamp` e reagrupados por dia para exibição, conforme visto na figura 22, nos cabeçalhos “terça-feira, 14 de julho” e “segunda-feira, 13 de julho”. Essa unificação é proposital: ao invés de apresentar quatro telas separadas por tipo de dado, o histórico busca reconstruir uma narrativa única do dia do usuário, aproximando-se do princípio de *Ecological Momentary Assessment* já mencionado anteriormente — o contexto de um registro de humor (por exemplo, o gatilho “Interno” registrado três minutos antes, na mesma figura) fica visualmente próximo do evento que possivelmente o motivou.

A propriedade `raw` armazena o dado original, permitindo que a listagem redirecione o usuário para a tela de detalhe correta ao tocar em um card, independentemente da categoria. Já a propriedade `badge` reaproveita o mesmo mecanismo já descrito na seção de indicadores: para um registro de sono, sinaliza se a duração foi insuficiente; para um registro de humor, destaca a dimensão mais relevante daquele registro (como “Ansiedade alta” na figura 22); para um gatilho, indica sua categoria, e assim por diante.

7.6 EXPORTAÇÃO DE DADOS E RELATÓRIOS

Além das telas de consulta e análise, o Nexo oferece duas formas de levar os dados para fora do aplicativo: a exportação integral dos registros, em formato aberto, e a geração de um relatório resumido em PDF. Ambas materializam o princípio de soberania dos dados discutido nos capítulos anteriores — princípio reforçado pelo questionário, em que a maioria dos respondentes atribuiu nota máxima à importância de ter controle total sobre as próprias informações. A diferença entre as duas reside no propósito: a exportação devolve a totalidade dos dados em um formato legível e processável, enquanto o relatório condensa um período específico em um documento pronto para ser guardado ou compartilhado com um profissional de saúde.

A exportação é acionada na tela de configurações, na opção “Exportar Meus

Dados". Antes de disparar o processo, o aplicativo pede uma confirmação e informa que o arquivo será enviado para o e-mail cadastrado — um lembrete explícito de que aquela operação retira dados do dispositivo e os entrega por um canal externo.

Do ponto de vista técnico, a solicitação enfileira um *job* assíncrono na mesma infraestrutura de filas descrita na seção 7.4. O *worker* percorre todas as entidades associadas ao usuário — perfil, humores e seus componentes, sono, gatilhos e seus vínculos, ações de cuidado, medicamentos, consultas, atividades e os indicadores já calculados — e serializa cada uma em um arquivo CSV próprio, com os rótulos traduzidos e os valores enumerados convertidos para texto legível. Os arquivos são compactados em um único *zip*, acompanhados de um manifesto com a data de geração, e o pacote é anexado a um e-mail enviado ao usuário; em caso de falha, um e-mail de erro é disparado em seu lugar. O resultado é uma exportação completa e portátil: nenhum campo é omitido ou agregado, de modo que o usuário pode reimportar os dados em outra ferramenta ou auditá-los diretamente — alinhado à exigência de portabilidade levantada nos requisitos não funcionais.

O relatório, por sua vez, é gerado sob demanda a partir da tela "Relatório", acessível também pelas configurações. O usuário escolhe um período — por meio de atalhos de 7, 30 ou 90 dias, ou um intervalo personalizado limitado a 90 dias — e aciona "Gerar relatório".

Figura 23: Tela de geração de relatório



Fonte: Elaborado pelo autor (2026)

O servidor reúne os registros de humor do período e os renderiza em um PDF intitulado “Relatório de Humor e Bem-Estar”, produzido com *@react-pdf/renderer* e devolvido ao aplicativo, que o oferece pela folha de compartilhamento nativa do sistema — permitindo salvá-lo ou enviá-lo a um terapeuta. O documento é dividido em duas páginas: a primeira traz uma visão geral (total de *check-ins*, humor predominante e dias rastreados), a distribuição percentual de cada humor e um mapa de calor diário; a segunda lista os sentimentos e emoções mais frequentes, com a intensidade média, e o registro detalhado dos *check-ins*, com os níveis de ansiedade, estresse e energia.

Cabe registrar, contudo, que o relatório desta versão cobre apenas os dados de humor. A intenção original era incorporar também o sono, as atividades e os padrões de gatilho — inclusive a correlação entre sono e energia já calculada nos indicadores —, mas essa ampliação foi adiada para uma versão futura. Como consequência, o

relatório atual complementa, mas não substitui, a exportação integral: para uma visão completa de todos os tipos de dado, o usuário ainda recorre ao *zip* de CSV.

8 TESTES

Este capítulo registra como o comportamento do Nexo é verificado. Como nos capítulos anteriores, o compromisso é com a honestidade: não se prescreve uma estratégia ideal, mas se descreve o estado da prática, o que cada tipo de teste é capaz — e incapaz — de detectar, e as lacunas assumidas explicitamente na Seção 8.6.

O foco recai sobre a verificação automatizada. A validação com usuários reais não foi realizada neste trabalho: o envolvimento de participantes em uma aplicação de saúde mental exigiria aprovação prévia de comitê de ética em pesquisa, levantamento de pessoas com transtorno de forma a aplicar com elas, cujo todo o trâmite não coube no cronograma do projeto. Essa ausência de avaliação externa é retomada como limitação na Seção 8.6.

Os dados quantitativos citados ao longo do capítulo referem-se ao estado do repositório em meados de outubro de 2026.

8.1 VOCABULÁRIO

Quatro termos delimitam o alcance dos testes descritos adiante: unitário, integração, *job* e *end-to-end* (E2E). Como os nomes e as fronteiras entre esses níveis variam entre autores, o Quadro 5 fixa o sentido adotado neste trabalho e o relaciona aos níveis usuais da literatura.

Tabela 5: Vocabulário de testes adotado neste trabalho

Termo	**O que exercita**	**Nível usual na literatura**
Teste unitário	Uma única função ou módulo, sem HTTP e sem banco de dados.	Teste de unidade.
Teste de integração	A aplicação HTTP real (rotas, validação, autenticação, controladores e serviços) sobre um banco de dados real; apenas o que sai do processo é simulado	Teste de integração, em nível de API.
Teste de *job*	Um processador de tarefa em segundo plano, chamado diretamente com uma carga fabricada, sobre o banco real.	Teste de integração de componente assíncrono
Teste *end-to-end* (E2E)	O aplicativo empacotado, em um dispositivo, ao longo de uma jornada real de usuário.	Teste de sistema.

Fonte: Elaborado pelo autor (2026)

8.2 ESTRATÉGIA DE VERIFICAÇÃO

O código-fonte do Nexo vive em um único repositório com dois projetos independentes: a API e o aplicativo móvel. O esforço de teste entre eles é deliberadamente assimétrico. A API possui uma suíte automatizada e é protegida por um portão de integração contínua; o aplicativo móvel é verificado majoritariamente de forma manual, com uma suíte *end-to-end* fina por cima. Essa assimetria não é um descuido, e se justifica por duas prioridades. A primeira é a capacidade de trabalho limitada: manter uma suíte automatizada completa nas duas esferas excederia o disponível, e concentra-se o esforço onde o retorno é maior. A segunda é que a API concentra a validação, a autenticação, as regras de negócio e a persistência — isto é, a garantia sobre a integridade dos dados registrados pelo usuário.

A Tabela 6 resume as ferramentas envolvidas e o ponto em que cada uma atua.

Tabela 6: Ferramentas de verificação

Finalidade	**Ferramenta**	**Onde atua**
Execução de testes e asserções da API	Vitest	API
Requisições HTTP nos testes	Supertest	Dentro dos arquivos do Vitest
Cobertura de código da API	Vitest (*coverage*)	API, sem limiar mínimo
Banco de dados de teste	PostgreSQL	*Container* local durante desenvolvimento; serviço efêmero no CI
Testes *end-to-end* móveis	Maestro	Dispositivo Android físico, via *adb*
Lint do aplicativo	ESLint (`expo lint`)	Manual, fora do CI
Integração contínua	GitHub Actions	API

Fonte: Elaborado pelo autor (2026)

8.3 SUÍTE DE TESTES DA API

A suíte automatizada concentra-se na API e cobre três frentes: os testes unitários, os testes de integração das rotas HTTP e os testes dos processadores de tarefas em segundo plano. No momento da redação, o diretório `api/tests` reunia 48 arquivos — 17 unitários, 26 de rotas e cinco de *jobs*. A execução e as asserções ficam a cargo do Vitest; as requisições HTTP são feitas pelo Supertest, que exercita a aplicação Express diretamente, sem abrir uma porta de rede, porém através de sua pilha HTTP real.

8.3.1 Organização e ferramentas

A configuração do Vitest (`api/vitest.config.ts`) divide a suíte em dois "projetos". O projeto `unit` seleciona os arquivos com o sufixo `.unit.test.ts` e não carrega configuração global alguma, de modo que esses testes dependem apenas das funções que exercitam. O projeto `db` seleciona os demais arquivos — integração e *job* —, aplica um arquivo de configuração global, fixa um limite de tempo por teste e faz com que esses testes trunquem e repovoem o banco.

A cobertura de código é apenas mensurada, não imposta: o relatório (`npm run test:coverage`) não possui limiar mínimo nem bloqueia a integração contínua. A verificação estática de tipos (`npm run typecheck`) também roda em caráter consultivo no CI — uma falha é registrada, mas não interrompe o *deploy*.

8.3.2 Execução e banco de dados de teste

Os testes que dependem do banco rodam contra uma instância real do PostgreSQL — e não contra um substituto em memória —, em uma base dedicada de nome sufixado com `_test`, criada por um pequeno script e materializada com `prisma db push`. O isolamento entre os casos é obtido por truncamento: antes de cada teste, uma rotina executa `TRUNCATE users CASCADE`, removendo a tabela de usuários e, por cascata, todas as dependentes. Não há *rollback* transacional nem banco efêmero por execução.

O carregamento de dados é feito por funções auxiliares escritas à mão, e não por uma biblioteca de fábricas: um módulo compartilhado (`api/tests/test-helper.ts`) oferece as primitivas centrais — cliente Prisma, limpeza da base, criação de usuário e geração de token —, e cada arquivo define os *seeders* de que necessita.

A regra para o uso de dublês na API é: simula-se apenas o que deixa o processo; a aplicação e o banco permanecem reais. A camada de filas (BullMQ) é substituída globalmente por um substituto que registra os *jobs* enfileirados, permitindo afirmar tanto a resposta HTTP quanto o trabalho em segundo plano agendado. O envio de arquivos e o cliente de armazenamento são substituídos por implementações controladas, e o disparo de e-mail e de notificações *push* é simulado pontualmente, nos arquivos que os exercitam.

Duas fronteiras permanecem deliberadamente reais: o acesso ao banco, via Prisma, nunca é simulado; tampouco a autenticação. Os testes geram tokens JWT válidos, assinados com o segredo de teste, e os enviam no cabeçalho de autorização, de modo que o *middleware* de autenticação real seja executado. Essa escolha reduz o risco de um teste passar porém se comportar de maneira distinta em produção.

8.3.3 Tipos de testes

A suíte da API divide-se em três grupos, retomando o vocabulário do Quadro 5: unitários, integração e tarefas assíncronas. Cada grupo é descrito a seguir com o que exercita, o que detecta e o que escapa ao seu alcance.

8.3.3.1 Testes unitários

Os testes unitários exercitam uma única função ou módulo, sem HTTP e sem banco de dados. Concentram-se na lógica pura do sistema, em grande parte extraída deliberadamente de seus envoltórios de persistência para se tornar testável: os agregadores dos indicadores, os construtores dos relatórios e auxiliares compartilhados, como paginação e conversão de agendamento. Também cobrem o gerador de código de ativação, a validade de tokens, os serializadores e as regras de validação. Por

serem rápidos e determinísticos, são o instrumento natural para fixar casos-limite — fronteiras de janelas de tempo, margens de classificação e aritmética de paginação — e servem como documentação executável dessas regras.

A limitação desse nível é estrutural: ele só alcança a lógica isolada em uma costura testável. Sem o banco e a camada HTTP, a maior parte do comportamento permanece fora de alcance, e o nível pouco contribui, isoladamente, para a confiança geral na aplicação.

8.3.3.2 Testes de integração

Os testes de integração formam a maior parte da suíte. Cada um constrói a aplicação real, conecta um cliente Prisma real e a exercita via Supertest, afirmando tanto a resposta HTTP quanto o estado persistido. Eles percorrem o caminho completo de uma requisição — roteamento, validação, autenticação, controladores, serviços e persistência — e, por operarem sobre um PostgreSQL real, não sofrem o descolamento que afeta os substitutos. Afirmar a resposta e as linhas persistidas simultaneamente verifica o efeito da operação, e não apenas o valor de retorno.

Em contrapartida, exigem infraestrutura em execução, são mais lentos e, por compartilharem a mesma base, rodam em série. O isolamento por truncamento é suficiente, porém menos rigoroso do que um *rollback* transacional, pois um teste que falha no meio pode deixar resíduos.

8.3.3.3 Testes de tarefas assíncronas

Os testes de *jobs* importam um processador do BullMQ e o chamam diretamente, com uma carga fabricada, contra o banco real. Cobrem o código que executa fora do ciclo requisição–resposta — os *workers* que geram exportações, enviam mensagens e recalculam indicadores —, chegando, no caso da exportação, a descompactar o anexo produzido e inspecionar seu conteúdo.

Como o processador é invocado diretamente, a maquinaria da fila em si — agendamento, repetição, tentativas e ordenação — permanece fora do alcance. A carga é fabricada à mão e, por isso, pode divergir daquela que o enfileirador realmente envia, sem que nada verifique uma contra a outra.

8.4 VERIFICAÇÃO DO APLICATIVO MÓVEL

Em linha com a assimetria descrita na Seção 8.2, o aplicativo móvel passou a contar com testes unitários em estágio inicial, ainda em andamento e sem a cobertura da suíte da API. A verificação cotidiana permanece predominantemente manual, em um dispositivo físico, contra a pilha local em execução — o aplicativo roda como *build*

de desenvolvimento do Expo, com dependências nativas do Android. Essa verificação captura o que os testes automatizados raramente alcançam — problemas visuais, de leiaute e de desempenho no dispositivo —, mas não é repetível, não protege contra regressões e permanece invisível à integração contínua.

8.4.1 Suíte *end-to-end* (Maestro)

Sobre a verificação manual há uma suíte *end-to-end* opcional, baseada no Maestro, ferramenta declarativa de automação de interface para dispositivos móveis. A suíte cobre uma única jornada central — login, registro de humor e saída — por três fluxos em YAML, marcados como `smoke` e executados por um pequeno *harness* (`frontend/scripts/e2e.sh`), exposto como `npm run test:e2e`. Os fluxos são dirigidos por identificadores de teste (`testID`) inseridos no código, e não pelo texto visível, o que os torna mais robustos a mudanças de interface.

A execução depende de um dispositivo Android físico conectado por `adb`, de um *build* de desenvolvimento, da API em funcionamento com um usuário de teste semeado e do Metro servindo o aplicativo; não há caminho sem cabeça nem integração ao CI, de modo que a suíte é executada manualmente e com pouca frequência. Cada fluxo limpa o estado do aplicativo e recarrega o *bundle*, tornando a execução lenta, e a cobertura permanece fina: um único caminho feliz, sem casos de erro. A suíte não está integralmente verde no momento da redação: os fluxos de login e de saída passam, mas o de registro de humor falha ao adicionar um componente de humor, porque a tela correspondente é abandonada pelo aplicativo pouco depois de ser aberta — causa ainda não identificada.

8.5 INTEGRAÇÃO CONTÍNUA

A integração contínua do Nexo (HUMBLE; FARLEY, 2011) é executada no GitHub Actions e testa somente a API, coerente com a estratégia assimétrica. O fluxo de implantação (`.github/workflows/deploy.yml`) dispara apenas quando há mudanças em `api`: sobe um PostgreSQL efêmero, instala as dependências, prepara a base de teste, executa a verificação de tipos em caráter consultivo e roda a suíte com o Vitest. A construção da imagem e a implantação em produção são condicionadas ao sucesso dessa etapa, de modo que uma suíte reprovada impede a publicação, conforme o Capítulo 6.4. O aplicativo móvel permanece fora da integração contínua: *lint* (`expo lint`) e verificação de tipos (`tsc -noEmit`) são executados apenas manualmente, e a suíte *end-to-end* não é disparada por fluxo algum.

8.6 COBERTURA E LACUNAS

Em síntese, a verificação automatizada cobre a superfície HTTP e a persistência da API, a sua lógica pura e os seus trabalhadores em segundo plano; a verificação do aplicativo móvel restringe-se, por ora, a uma jornada manual de fumaça e a testes unitários ainda em consolidação. Permanecem pouco cobertas a lógica do *frontend* entre o nível de unidade e o *end-to-end*, a maquinaria da fila em si (agendamento, repetições e ordenação) e os caminhos de erro do aplicativo.

Persistem, ainda, três limitações que merecem registro. A primeira é a ausência de validação com usuários reais, motivada pelo trâmite de comitê de ética em pesquisa descrito na abertura deste capítulo: as especificações automatizadas atestam o comportamento programado, mas não a adequação do sistema às expectativas do público-alvo. A segunda é a cobertura de código medida, mas não imposta, o que orienta o trabalho sem jamais reprovar uma *build* ou *deploy*. A terceira são os riscos de descolamento já mencionados: os dados de teste escritos à mão em cada arquivo e a cópia espelhada dos tipos da API mantida no *frontend* podem divergir da fonte de verdade sem que nenhum teste acuse. Essas lacunas são assumidas de forma explícita e refletem uma escolha de priorização, não uma garantia de ausência de defeitos, e que podem ser corrigidas em incrementos no futuro.

9 CONSIDERAÇÕES FINAIS

Este trabalho partiu de uma lacuna identificada na interseção entre o autorregistro de humor e o cotidiano de pessoas com Transtorno Afetivo Bipolar: embora o *Ecological Momentary Assessment* ofereça um caminho consolidado para reduzir os vieses de memória e aproximar o registro do momento em que a experiência ocorre, os aplicativos disponíveis frequentemente fragmentam sono, humor, gatilhos e ações de cuidado em telas isoladas, restringem a exportação dos dados e operam como soluções fechadas, com pouca transparência sobre o uso das informações sensíveis que coletam.

Para responder a essa lacuna, foi desenvolvido o Nexo, uma aplicação móvel de código aberto que integra esses registros em uma única linha do tempo e produz indicadores destinados a auxiliar o usuário na percepção de padrões ao longo do tempo. O sistema é composto por uma API REST e por um aplicativo móvel, ambos escritos em TypeScript, com processamento de indicadores em segundo plano, implantação automatizada e uma suíte de testes concentrada na API, conforme descrito ao longo dos capítulos anteriores.

A pesquisa aplicada de natureza exploratória orientou as decisões de projeto. O questionário aplicado a 48 potenciais usuários, apresentado no Capítulo 3, e a análise de aplicativos similares forneceram os insumos que sustentaram a definição dos requisitos, da identidade visual e da organização do sistema, bem como a adoção do modelo incremental com princípios ágeis e *Kanban* no desenvolvimento.

Diante disso, conclui-se que os objetivos propostos foram alcançados: o protótipo funcional entregue demonstra a viabilidade de uma solução de autorregistro alinhada aos princípios de privacidade, transparência e autonomia do usuário sobre os próprios dados.

O trabalho, contudo, apresenta limitações que devem ser explicitadas. A verificação do aplicativo móvel permanece majoritariamente manual e com uma suíte *end-to-end* fina, de modo que regressões na interface podem ser incorporadas ao código sem que nenhum teste acuse, como discutido no Capítulo 8. A amostra do questionário é, por conveniência, e não pode ser generalizada; além disso, o protótipo ainda não foi submetido a uma avaliação com usuários reais, o que deixa em aberto a confirmação de que as escolhas de interface e de tom atendem, de fato, ao

público-alvo. Por fim, algumas decisões sensíveis — como o rótulo de estabilidade apresentado nos indicadores — permanecem deliberadamente estáticas, aguardando embasamento específico antes de se tornarem funcionais.

Os resultados obtidos abrem caminhos de continuidade que, na leitura deste trabalho, merecem prioridade. O primeiro é a reaplicação da pesquisa junto a usuários potenciais: repetir o questionário descrito no Capítulo 3 — ou uma versão adaptada dele — em uma nova amostra, desta vez acompanhada de um teste do protótipo funcional, permitiria comparar as expectativas registradas em 2025 com a percepção concreta do aplicativo, avaliando usabilidade, tom de comunicação e interesse pelas funcionalidades, e capturando ajustes antes de uma eventual implantação mais ampla.

Uma segunda frente é a criptografia no lado do cliente para anotações e registros de humor, de modo que o conteúdo sensível seja cifrado antes de deixar o dispositivo e permaneça inacessível mesmo ao operador do servidor. Essa medida responde diretamente à preocupação com privacidade registrada no questionário, em que a maioria dos respondentes atribuiu nota máxima à importância do controle sobre os próprios dados, e reforça o cenário de auto-hospedagem descrito no Capítulo 6.4.

A terceira frente é ampliar a testabilidade do aplicativo móvel, hoje verificado de forma predominantemente manual (Seção 8.4). Extrair a lógica de apresentação e de agregação em camadas testáveis, adotar testes de componente e integrar *lint*, verificação de tipos e a suíte *end-to-end* ao fluxo de integração contínua reduziria o risco de regressões invisíveis e aproximaria o nível de confiança do *frontend* àquele já alcançado pela API.

Além dessas, as lacunas apontadas na Seção 8.6 — como a maquinaria da fila fora do alcance dos testes e a duplicação manual dos dados de teste — sugerem ajustes de menor porte, porém de impacto relevante na manutenibilidade do projeto a médio prazo.

REFERÊNCIAS

ANDERSON, David J. **Kanban**: successful evolutionary change for your technology business. [S.l.]: Blue Hole Press, 2010.

ANDERSSON, Rasmus. **Inter font family**. Disponível em: <https://rsms.me/inter/>.

Associação Brasileira de Transtorno Bipolar (ISBD Brazil). **Associação Brasileira de Transtorno Bipolar**. Acessado em 13 jul. 2026. Disponível em: <https://isbdbrazil.org>.

COELHO, Beatriz. **Metodologia Científica**: O que é e como fazer - Guia completo. Disponível em: <https://blog.mettzer.com/metodologia-cientifica/>.

Colin McDonnell. **Zod**: TypeScript-First Schema Validation. Acessado em 13 jul. 2026. Disponível em: <https://zod.dev>.

COLLINS, Lorraine; KASHDAN, Todd B.; GOLLNISCH, Gernot. The feasibility of using cellular phones to collect ecological momentary assessment data: Application to alcohol consumption. **Experimental and Clinical Psychopharmacology**, [S.l.], v.11, n.1, p.73–78, 2003.

Docker, Inc. **Docker**. Acessado em 13 jul. 2026. Disponível em: <https://www.docker.com>.

Docker, Inc. **Docker Compose**. Acessado em 13 jul. 2026. Disponível em: <https://docs.docker.com/compose/>.

EBNER-PRIEMER, Ulrich W.; TRULL, Timothy J. Ecological momentary assessment of mood disorders and mood dysregulation. **Psychological Assessment**, [S.l.], v.21, n.4, p.463–475, 2009.

EVANS, Joel R.; MATHUR, Anil. The value of online surveys. **Internet Research**, [S.l.], v.15, n.2, p.195–219, 2005.

FAURHOLT-JEPSEN, Maria et al. Electronic self-monitoring of mood using it platforms in adult patients with bipolar disorder: A systematic review of the validity and evidence. **BMC Psychiatry**, [S.l.], v.16, n.1, Jan 2016.

FOWLER, Martin. **Patterns of enterprise application architecture**. Nineteenth printing.ed. Boston San Francisco New York Toronto Montreal London Munich Paris Madrid Capetown: Addison-Wesley, 2013. (The Addison-Wesley Signature Series).

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design patterns**: elements of reusable object-oriented software. 39. printing.ed. Boston, Mass. Munich: Addison-Wesley, 2011. (Addison-Wesley professional computing series).

GERHARDT, Tatiana Angel; SILVEIRA, Denise Tolfo. **Métodos de Pesquisa**. 1.ed. [S.l.]: Editora da UFRGS, 2009.

GitHub, Inc. **GitHub Actions**. Acessado em 13 jul. 2026. Disponível em: <https://github.com/features/actions>.

GOOGLE. **Typography – Material Design 3**. [S.l.]: Google, 2021. Disponível em: <https://m3.material.io/styles/typography/overview>.

GUEDES, Gilleanes T. A. **UML 2**: uma abordagem prática. [S.l.]: Novatec Editora, 2022.

HARDT, D. **The OAuth 2.0 Authorization Framework**. [S.l.: s.n.], 2012. n.RFC 6749.

HUCKVALE, Kit; TOROUS, John; LARSEN, Mark E. Assessment of the Data Sharing and Privacy Practices of Smartphone Apps for Depression and Smoking Cessation. **JAMA Network Open**, [S.l.], v.2, n.4, p.e192542, 2019.

HUMBLE, Jez; FARLEY, David G. **Continuous delivery**: reliable software releases through build, test, and deployment automation. 7th printing 2013.ed. Upper Saddle River, NJ: Addison-Wesley, 2011. (A Martin Fowler Signature Book).

IBM Documentation. **Modelando a Estrutura Estática Utilizando Diagramas de Classes**. Disponível em: <https://www.ibm.com/docs/pt-br/rsas/7.5.0?topic=model-modeling-class-diagram-static-structure>. Acesso em: 2026-07-03.

IEEE. IEEE Standard Glossary of Software Engineering Terminology. **IEEE Std 610.12-1990**, [S.l.], p.1–84, Dec. 1990.

IWAYA, Leonardo Horn; BABAR, M. Ali; RASHID, Awais; WIJAYARATHNA, Chamila. On the privacy of mental health apps. **Empirical Software Engineering**, [S.l.], v.28, n.1, Nov 2022.

LAKATOS, Eva Maria; MARCONI, Marina de Andrade. **Fundamentos de Metodologia Científica**. 9.ed. SÃO PAULO, SP: Editora Atlas Ltda, 2022.

LIDWELL, W.; HOLDEN, K.; BUTLER, J. **Universal Principles of Design**. 2.ed. Gloucester, Massachusetts, USA: Rockport Publishers, 2011.

MARTIN, Robert C. **Clean code**: a handbook of agile software craftsmanship. Repr..ed. Upper Saddle River, NJ Munich: Prentice Hall, 2012. (Robert C. Martin series).

MELTON, Bridget F.; BIGHAM, Lauren E.; BLAND, Helen W. The feasibility of using video journaling to collect ecological momentary assessment data: application to health behavior change interventions. **Journal of Computing in Higher Education**, [S.l.], v.25, n.1, p.12–26, 2013.

MERIKANGAS, Kathleen R. et al. Prevalence and Correlates of Bipolar Spectrum Disorder in the World Mental Health Survey Initiative. **Archives of General Psychiatry**, [S.l.], v.68, n.3, p.241–251, 3 2011.

Meta Platforms, Inc. **React Native**. Acessado em 13 jul. 2026. Disponível em: <https://reactnative.dev>.

MORTON, Emma et al. Evaluating the quality, safety, and functionality of commonly used smartphone apps for bipolar disorder mood and sleep self-management. **International Journal of Bipolar Disorders**, [S.l.], v.10, n.1, p.10, 2022.

OpenJS Foundation. **Express — Node.js Web Application Framework**. Acessado em 13 jul. 2026. Disponível em: <https://expressjs.com>.

Poimandres. **Zustand**. Acessado em 13 jul. 2026. Disponível em: <https://zustand.docs.pmnd.rs>.

PostgreSQL Global Development Group. **PostgreSQL**: The World's Most Advanced Open Source Relational Database. Acessado em 13 jul. 2026. Disponível em: <https://www.postgresql.org>.

PRESSMAN, Roger S. **Engenharia de software**: uma abordagem profissional. 9.ed. Porto Alegre, RS: Amgh, 2021.

Prisma Data, Inc. **Prisma**. Acessado em 13 jul. 2026. Disponível em: <https://www.prisma.io>.

Red Hat, Inc. **Ansible**. Acessado em 13 jul. 2026. Disponível em: <https://www.ansible.com>.

RYAN, Kelly Ann et al. A Smartphone App to Monitor Mood Symptoms in Bipolar Disorder: Development and Usability Study. **JMIR Ment Health**, [S.l.], v.7, n.9, p.e19476, Sep 2020.

SHAFRANOVICH, Y. **Common Format and MIME Type for Comma-Separated Values (CSV) Files**. [S.l.: s.n.], 2005. n.RFC 4180.

SOMMERVILLE, Ian. **Engenharia de software**. [S.l.]: Addison Wesley, 2010.

Taskforce.sh Inc. **BullMQ**. Acessado em 13 jul. 2026. Disponível em: <https://bullmq.io>.

Valkey Contributors, Linux Foundation. **Valkey**. Acessado em 13 jul. 2026. Disponível em: <https://valkey.io>.

APÊNDICES

APÊNDICE A – DOCUMENTAÇÃO DE CASOS DE USO

Nome do Caso de Uso	UC01 — Login
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite que o usuário realize a autenticação no sistema.
Pré-condições	O usuário deve possuir uma conta cadastrada.
Pós-condições	Usuário autenticado e direcionado ao painel inicial.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Informa e-mail e senha.	
	2. O sistema valida as credenciais.
	3. O sistema cria a sessão autenticada.
	4. O sistema apresenta o painel inicial.
Restrições e validações	O e-mail deve estar cadastrado.
	A senha deve estar correta.
	A conta deve estar ativada.
Cenário Alternativo A1 — Primeiro acesso	
Ações do Ator	**Resposta do Sistema**
1. O usuário realiza o primeiro login.	
	2. O sistema solicita a ativação da conta.
Cenário de Exceção E1 — Credenciais inválidas	
Ações do Ator	**Resposta do Sistema**
1. O usuário informa dados inválidos.	
	2. O sistema apresenta mensagem de erro e solicita nova tentativa.

Nome do Caso de Uso	UC02 — Cadastro
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite que um novo usuário crie uma conta para utilizar o sistema.
Pré-condições	O usuário não deve possuir cadastro prévio utilizando o mesmo endereço de e-mail.
Pós-condições	Conta criada com sucesso e apta para ativação.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Seleciona a opção de cadastro.	
	2. O sistema apresenta o formulário de cadastro.
3. Informa nome, e-mail e senha.	
	4. O sistema valida os dados informados.
	5. O sistema cria a conta do usuário.
	6. O sistema solicita a ativação da conta.
Restrições e validações	O endereço de e-mail deve ser único.
	A senha deve atender aos critérios mínimos de segurança.
Cenário de Exceção E1 — E-mail já cadastrado	
Ações do Ator	**Resposta do Sistema**
1. O usuário informa um e-mail existente.	
	2. O sistema informa que já existe uma conta cadastrada.

Nome do Caso de Uso	UC03 — Consultar Registros de Humor
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite consultar os registros recentes de humor e sono, bem como os indicadores consolidados referentes aos níveis médios de energia e sono do usuário.
Pré-condições	Usuário autenticado.

Pós-condições	Os registros e indicadores são apresentados ao usuário.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Acessa o painel inicial.	
	2. O sistema recupera os registros de humor recentes.
	3. O sistema recupera informações relacionadas ao sono e energia.
	4. O sistema apresenta os registros e indicadores ao usuário.
Restrições e validações	Somente registros pertencentes ao usuário autenticado podem ser consultados.
	É necessário possuir sessão autenticada.
Cenário Alternativo A1 — Não existem registros	
	O sistema apresenta uma mensagem informando que ainda não existem registros cadastrados.

Nome do Caso de Uso	UC04 — Manter Humor
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite registrar, editar e remover registros de humor.
Pré-condições	Usuário autenticado.
Pós-condições	O registro de humor é armazenado e passa a compor o histórico do usuário.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Seleciona registrar humor.	2. O sistema apresenta o formulário de registro.
3. Informa o humor principal.	
4. Informa os níveis emocionais e demais informações.	
5. Adiciona observações, caso desejar.	
6. Confirma o registro.	

	7. O sistema valida os dados informados.
	8. O sistema registra as informações.
	9. Sistema agenda a atualização dos indicadores e da correlação entre sono e energia.
Restrições e validações	O humor principal é obrigatório.
	Os níveis emocionais devem estar dentro do intervalo permitido.
	A data do registro é obrigatória.
Cenário Alternativo A1 — Vincular gatilho	
Ações do Ator	**Resposta do Sistema**
1. Seleciona um ou mais gatilhos.	
	2. O sistema associa os gatilhos ao registro de humor.

Nome do Caso de Uso	UC05 — Manter Gatilhos
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite cadastrar, editar e remover gatilhos emocionais que ocorreram naquele dia e que poderão ser associados aos registros de humor e consultas.
Pré-condições	Usuário autenticado.
Pós-condições	O gatilho é salvo e passa a compor o histórico do usuário.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Seleciona registrar gatilho.	
	2. O sistema apresenta o formulário.
3. Informa uma descrição do gatilho e seleciona um horário.	
	4. O sistema valida os dados informados.
	5. O sistema salva o novo gatilho.
	6. O sistema apresenta uma mensagem de confirmação.
Restrições e validações	O horário e a categoria do gatilho é obrigatório.
Cenário de Exceção E1 — Horário inválido	

1. Informa um horário inválido.	
	2. O sistema informa que o horário é inválido.

Nome do Caso de Uso	UC06 — Manter Sono
Ator Principal	Usuário
Atores Secundários	Sistema (processamento em segundo plano)
Resumo	Permite registrar, editar e remover registros de sono do usuário.
Pré-condições	Usuário autenticado.
Pós-condições	O registro de sono é armazenado e os indicadores relacionados são atualizados.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Seleciona registrar sono.	
	2. O sistema apresenta o formulário de registro.
3. Informa a data do registro.	
4. Informa uma média de horas de sono.	
5. Informa uma anotação, caso desejar.	
6. Confirma o registro.	
	7. O sistema valida os dados informados.
	8. O sistema registra o período de sono.
	9. O sistema agenda o cálculo da média diária de sono.
	10. O sistema atualiza os indicadores apresentados ao usuário.
Restrições e validações	A data do registro é obrigatória.

Nome do Caso de Uso	UC07 — Manter Ações de Cuidado
Ator Principal	Usuário
Atores Secundários	—

Resumo	Permite registrar ações de cuidado relacionadas ao acompanhamento da saúde do usuário, como medicamentos, atividades, consultas e tomadas de medicamentos.
Pré-condições	Usuário autenticado.
Pós-condições	A ação de cuidado é registrada e passa a compor o histórico do usuário.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Seleciona registrar ação de cuidado.	
	2. O sistema apresenta os tipos de ações disponíveis.
3. Seleciona o tipo desejado.	
	4. O sistema apresenta o formulário correspondente.
5. Informa os dados necessários.	
	6. Confirma o registro.
	7. O sistema valida os dados informados.
	8. O sistema registra a ação de cuidado.
	9. O sistema atualiza os registros recentes.
Restrições e validações	Os campos obrigatórios dependem do tipo de ação selecionada.
	Somente ações válidas podem ser registradas.
Cenário Alternativo A1 — Registrar Consulta	
Ações do Ator	**Resposta do Sistema**
1. Seleciona vincular um registro de humor ou gatilho.	
	2. O sistema encaminha para a tela de registro do vinculo escolhido.
Cenário Alternativo A2 — Registrar Tomada de Medicamento	
1. Seleciona registrar uma tomada de medicamento.	
2. Seleciona um horário.	
	3. O sistema registra a tomada do medicamento.
Cenário de Exceção E1 — Não existem medicamentos cadastrados	

1. Seleciona registrar uma tomada de medicamento.	
	2. O sistema informa que não existem medicamentos cadastrados para registro de tomada.

Nome do Caso de Uso	UC08 — Visualizar Histórico
Ator Principal	Usuário
Atores Secundários	—
Resumo	Permite consultar o histórico de registros de humor, sono e ações de cuidado agrupados por data.
Pré-condições	Usuário autenticado.
Pós-condições	Os registros históricos são apresentados ao usuário.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
1. Acessa o histórico.	
	2. O sistema recupera os registros do usuário.
	3. O sistema agrupa os registros por data.
	4. O sistema apresenta indicadores resumidos para cada registro.
5. Seleciona um registro.	
	6. O sistema apresenta os detalhes do registro selecionado.
7. Seleciona editar ou remover o registro.	
	8. O sistema verifica se o período permitido para alteração ainda é válido.
	9. O sistema permite a edição ou remoção do registro.
Restrições e validações	Somente registros do próprio usuário podem ser consultados.
	Registros somente podem ser editados ou removidos durante os primeiros cinco minutos após sua criação.

Cenário Alternativo A1 — Histórico vazio	
1. Acessa o histórico.	
	2. O sistema informa que não existem registros cadastrados.
Cenário de Exceção E1 — Tempo para edição expirado	
1. Seleciona editar um registro expirado.	
	2. O sistema informa que o período permitido para alteração foi encerrado.

Nome do Caso de Uso	UC09 — Calcular Correlação Sono × Energia
Ator Principal	Sistema
Atores Secundários	—
Resumo	Calcula automaticamente a correlação entre os registros recentes de sono e os níveis de energia informados pelo usuário.
Pré-condições	Existem registros suficientes para o cálculo da correlação.
Pós-condições	O indicador de correlação é atualizado.
Cenário Principal	
Ações do Ator	**Resposta do Sistema**
	1. O sistema identifica a criação ou atualização de um registro de humor ou sono.
	2. O sistema recupera os registros correspondentes aos últimos sete dias.
	3. O sistema calcula a correlação utilizando o coeficiente de Pearson.
	4. O sistema atualiza o indicador apresentado ao usuário.
Restrições e validações	O cálculo considera apenas registros dos últimos sete dias.
	É necessário existir quantidade mínima de registros para o cálculo estatístico.
Cenário Alternativo A1 — Dados insuficientes	

	O sistema mantém o indicador indisponível até que existam registros suficientes para o cálculo.

APÊNDICE B – RESULTADOS DA PESQUISA

Nesse apêndice estão contidas os dados referentes as respostas de maior relevância para a implementação do sistema. Considerando principalmente que ela possui o total de vinte perguntas.[1]

1. Você já teve algum interesse ou experiência com o monitoramento do seu humor ou estado emocional?		
	%	**TOTAL**
Sim, faço regularmente	18,8	9
Sim, já tentei alguma vez	20,8	10
Tenho interesse, mas nunca tentei	52,1	25
Não tenho interesse e nunca tentei	8,3	4

Fonte: Elaborado pelo autor (2026)

2. Respostas sobre a faixa etária dos participantes		
	%	**TOTAL**
Menos de 18 anos	0	0
Entre 18 e 30 anos	43,8	21
31 a 45 anos	29,2	14
46 a 60 anos	22,9	11
Mais de 60 anos	4,2	2

Fonte: Elaborado pelo autor (2026)

3. Com qual gênero você se identifica?		
	%	**TOTAL**
Feminino	66,7	32
Masculino	33,3	16
Não-binário	0	0
Prefiro não dizer	0	0

Fonte: Elaborado pelo autor (2026)

[1] Questão de múltipla escolha (respostas não somam 100%). Apenas foi contabilizado os 29 respondentes que relataram ter monitorado o humor de alguma forma.

3. Você já recebeu algum diagnóstico relacionado à saúde mental?		
	%	TOTAL
Sim	37,5	18
Não	62,5	30
Prefiro não dizer	0	0

Fonte: Elaborado pelo autor (2026)

Você realiza algum acompanhamento psicológico ou psiquiátrico atualmente?		
	%	TOTAL
Sim, com psicólogo(a)	12,5	6
Sim, com psiquiatra	4,2	2
Sim, com ambos	12,5	6
Não	70,8	34

Fonte: Elaborado pelo autor (2026)

Você já monitorou seu humor em algum momento da vida?		
	%	TOTAL
Sim, regularmente	21,3	10
Sim, de vez em quando	12,8	6
Já tentei, mas parei	21,3	10
Nunca	44,7	21

Como você costuma ou costumava fazer esse monitoramento?		
	%	TOTAL
Apenas mentalmente	48,3	14
Diário escrito	37,9	11
Aplicativos	34,5	10
Afetivograma	10,3	3
Planilhas	6,9	2
Auto observação	3,4	1

Se você parou ou teve dificuldades, quais foram os motivos?		
	%	TOTAL
Esquecimento	56,0	14
Falta de tempo	40,0	10
Preocupações com privacidade	24,0	6
Não percebi benefícios	20,0	5
Ferramentas difíceis de usar	8,0	2
Dificuldade de analisar sozinho	4,0	1

Quão importante é para você ter controle total sobre seus dados pessoais no aplicativo?		
	%	TOTAL
Muito importante	68,1	32
Importante	12,8	6
Neutro	12,8	6
Pouco importante	2,1	1
Nada importante	4,3	2

Que tipo de tom você preferiria em um aplicativo de humor?		
	%	**TOTAL**
Acolhedor e empático	37,5	18
Neutro e objetivo	25,0	12
Clínico e profissional	20,8	10
Lúdico e leve	12,5	6
Moderno e minimalista	2,1	1
Não gosto de apps que parecem um formulário psiquiátrico	2,1	1

Para além dos itens citados acima, no formulário existia a opção de deixar uma contribuição, essa lista apresenta as sugestões de funcionalidades adicionais obtidas a partir das respostas dos participantes na questão de entrada livre:

- Questões hormonais e voltadas para mulheres que acompanham as mudanças de humor no ciclo menstrual.
- Seria legal ver um gráfico semanal ou mensal do meu humor.
- Algo totalmente offline ou que eu possa apagar quando quiser sem me preocupar.
- Sugestões de atividades ou conteúdos com base no humor registrado (música, respiração guiada, frases motivacionais, pequenas metas diárias).
- Design que considere a acessibilidade para pessoas neuro divergentes (cores e tipografia suaves).
- Opção de visualizar gráficos ao longo do tempo.
- Bloco de notas para pensamentos, reflexões e desabafos.
- Função para considerar o ciclo menstrual no humor.
- Possibilidade de escrita, registro de sintomas físicos, sugestões de regulação.

Devido à natureza livre das respostas, esses dados foram normalizados e removidos duplicatas ou elementos julgados nulos (respostas como “nada a acrescentar”, etc).